All materials

SIAM LS Minitutorial #1
Interpretable Machine Learning and Deep Learning for Biological Modeling

Suzanne Fernandes-Sindi (UC Merced)
Tracey Oellerich (UNC Chapel Hill)

–

Monday, July 6 (8:00 - 10:00 am)
Grand Ballroom C

UCMERCED

THE UNIVERSITY of NORTH CAROLINA at CHAPEL HILL

Code of Conduct

Do

- Be respectful in speech and actions
- Be mindful of surroundings and fellow participants
- Notify organizers of dangerous situations, distress, etc., even if they seem inconsequential.

Don't

- Demean, discriminate, or ha rass, humiliate, intimidate, stalk, disrupt,
- Assault, inappropriately touch

Minitutorial Schedule

SIAM LS Minitutorial #1

https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • Preventing Overfitting • Ex #2: Application
50 Mins	Deep Learning
	• Definitions • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Overview & Orientation

SIAM LS Minitutorial #1

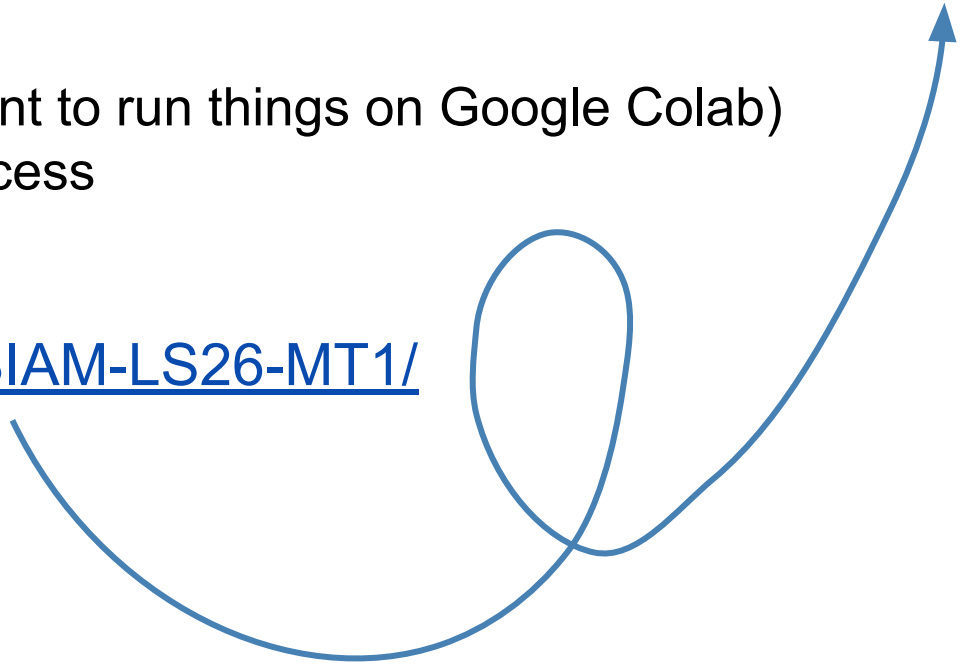
https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	**Overview & Orientation**
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • Preventing Overfitting • Ex #2: Application
50 Mins	Deep Learning
	• Definitions • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Overview

- What will you need?
  - Google Account (if you want to run things on Google Colab)
  - Computer with internet access

Our GitHub:
https://github.com/ssindiUCM/SIAM-LS26-MT1/



What is a Jupyter Notebook

A **Jupyter Notebook** is an interactive document used for writing and running code while also including explanations and results in one place.

It combines:

- Code (e.g., Python)
- Text & notes
- Outputs like tables, plots, and figures

It is commonly used for:

- Learning programming and data science
- Analyzing and visualizing data
- Sharing reproducible research and results

Think of it as a digital lab notebook where you can write code, run it, and immediately see and explain the results, all in the same file.

What is Google Colab

Cloud-based notebook for Python

Why use it?

- 🌐 Runs in your browser
- ☁️ Connects to Google Drive
- ⚙️ No installation required
- 🤝 Easy to share and collaborate
- 📊 Great for data analysis and machine learning

Requirements:

- Google account,
- internet access
- a web browser

How to import a GitHub Repository into Colab:

1. Open a Colab notebook
2. Mount Google Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. Change to your working folder:

```
%cd /content/drive/MyDrive/
```

4. Clone the GitHub repo:

```
!git clone
https://github.com/username/repository-name
```

5. Move into the repo folder:

```
%cd repository-name
```

6. Start coding

Basic Definitions

Machine Learning is the process of fitting flexible mathematical models to data in order to make predictions, identify structure, or discover relationships.

Examples: Linear regression; dimensionality reduction; fitting parameters in an ODE

Interpretability refers to how easily humans can understand the relationship between inputs, outputs, and model behavior.

Examples: Linear Regression (Intercept, Coefficients)

Workshop Learning Objectives

Part 1: Traditional Machine Learning

- Understand the interpretability–accuracy trade-off
- Select appropriate ML methods for biological data
- Recognize and avoid common machine learning pitfalls in biological modeling

Part 2: Deep Learning

- Understand the core language with deep-learning
- Understand the interpretability spectrum within deep-learning
- See where deep-learning meets mechanistic modeling.

Traditional Machine Learning

SIAM LS Minitutorial #1

https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• **Definitions** • Ex #1: Binary Classification • Preventing Overfitting • Ex #2: Application
50 Mins	Deep Learning
	• Definitions • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

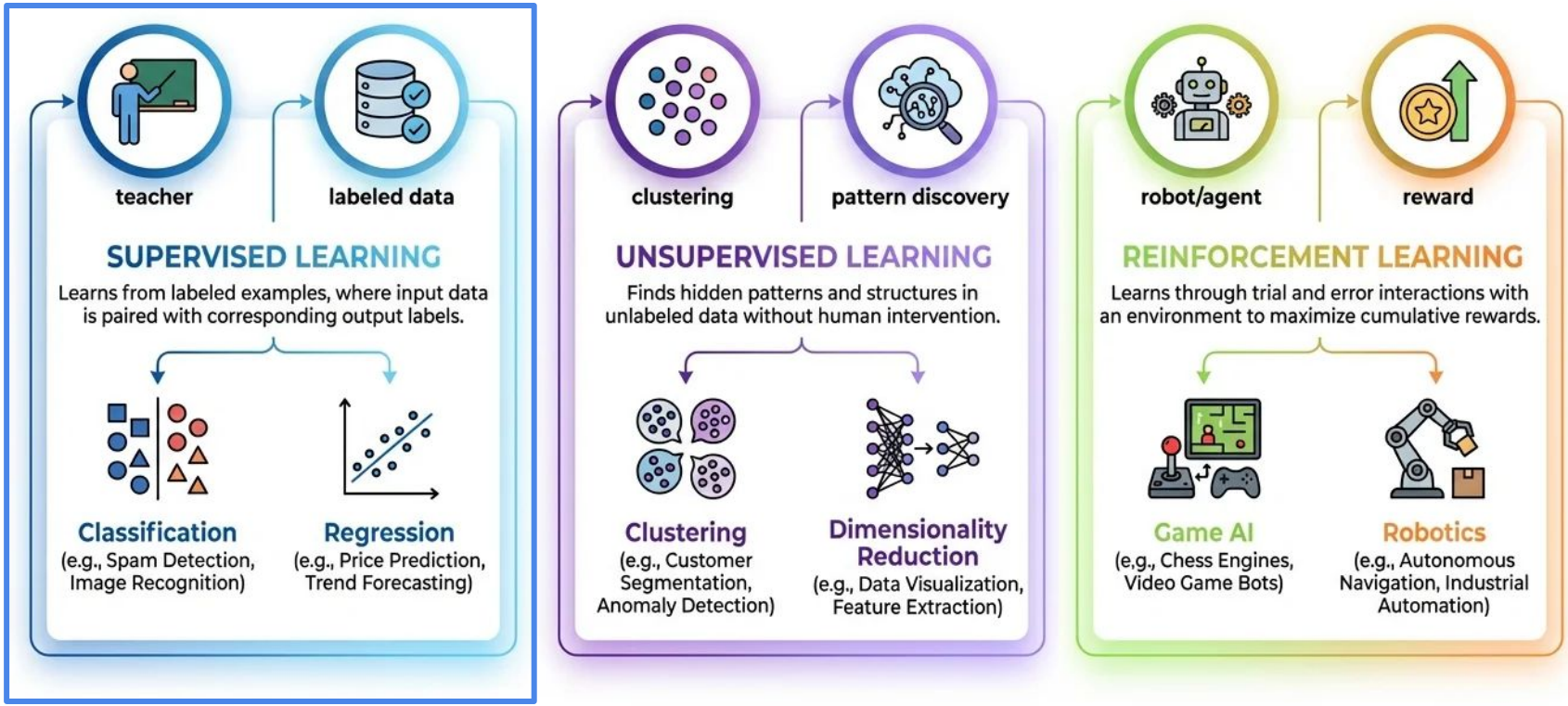
Machine Learning (ML) Overview

Machine Learning is the process of fitting flexible mathematical models to data in order to make predictions, identify structure, or discover relationships.

Key ideas

- Learn patterns from data rather than explicitly programming rules.
- Generalize those patterns to new, unseen data.

Types of Machine Learning



Core Building Blocks of Machine Learning

Model

Mathematical algorithm that learns patterns from the data

Examples: Logistic Regression, Random Forest

Inputs (Features, x)

Measurements used by the model

Examples: age, sex, protein abundances, etc

Output (Prediction, y)

The quantity the model is trying to predict
Depends on the machine learning task

Examples: Classification: binary label
Regression: Predicted biomarker level

Traditional Machine Learning

SIAM LS Minitutorial #1

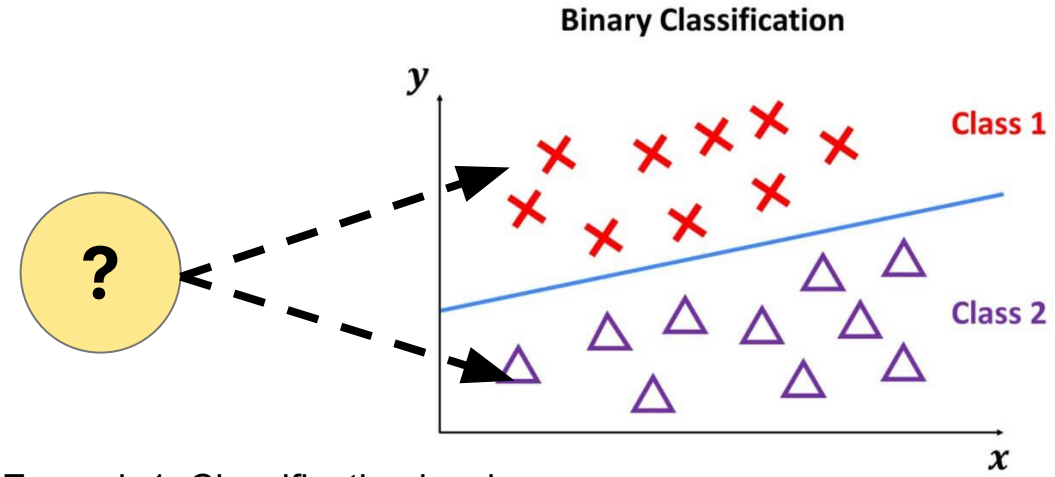
https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • **Ex #1: Binary Classification** • Preventing Overfitting • Ex #2: Application
50 Mins	Deep Learning
	• Definitions • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Example 1: Binary Classification

1) What machine learning model to select?
2) How do you train a machine learning classifier?
3) How do you get interpretable information from model?

Goal: Identify predictive feature combinations without overfitting



Python notebook: ML_Example1_Classification.ipynb

Example 1: Classification Data Characteristics

Data

N = 1000 synthetic data points generated using `make_classification`

Inputs (Features)

n = 5 features per individual

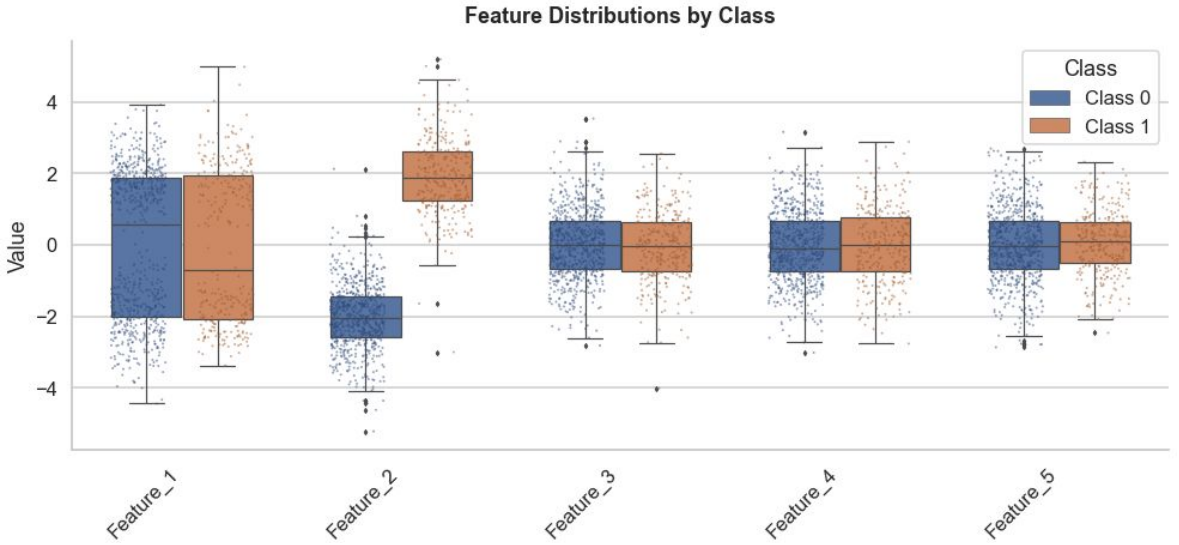
Output (Prediction, y)

Binary class label

Note: The data has a disproportionate class distribution (70/30).

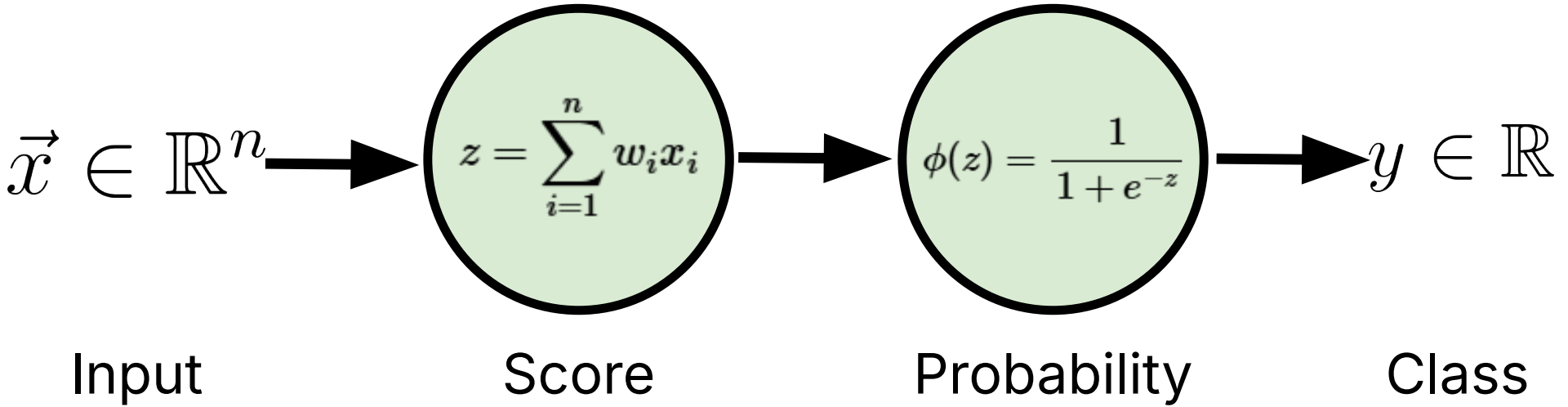
Example 1: Classification Data Characteristics

- Feature 2 appears to separate the two classes very well.
- Feature 1 shows substantial overlap, so it may be less useful for classification.
- Although individual features overlap, their combination may still help separate the classes.



Classification: Logistic Regression

Logistic regression is a supervised machine learning algorithm that predicts the probability that an given data point belongs to a category/class.



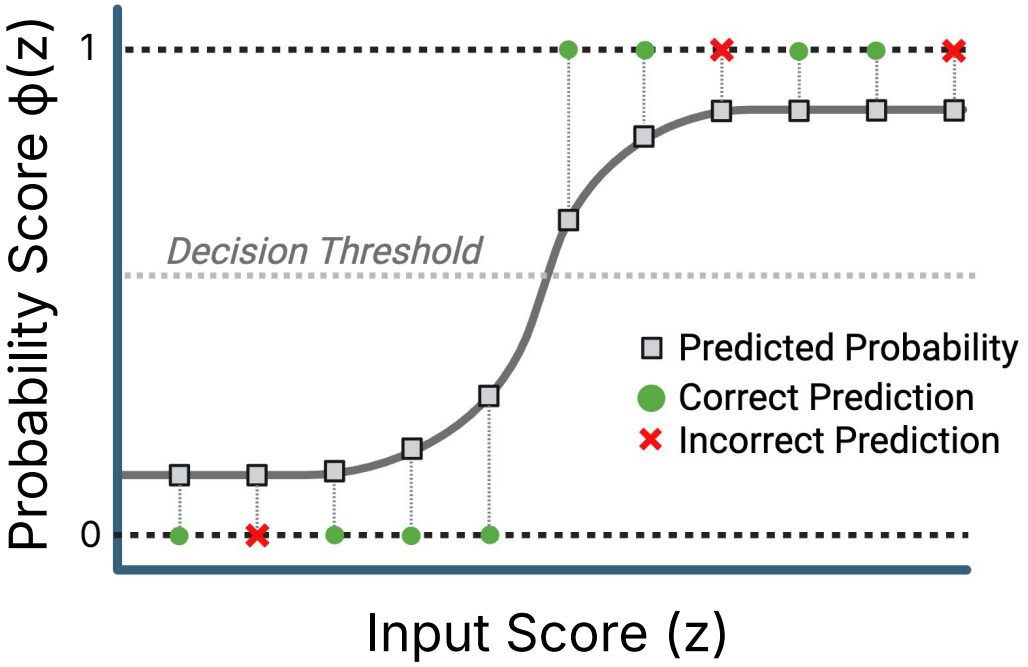
Classification: Logistic Regression

Logistic regression is a supervised machine learning algorithm that predicts the probability that an given data point belongs to a category/class.

$$z = \sum_{i=1}^{n} w_i x_i$$

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

$$y = \begin{cases} 0 & \phi(z) < \text{threshold} \\ 1 & \phi(z) > \text{threshold} \end{cases}$$



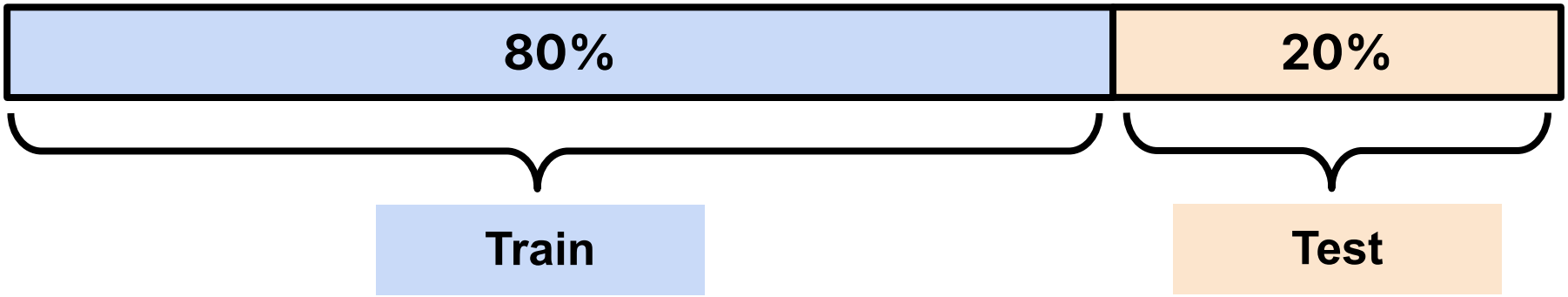
Classification: Logistic Regression Interpretability

$$z = \sum_{i=1}^{n} w_i x_i \qquad \phi(z) = \frac{1}{1 + e^{-z}}$$

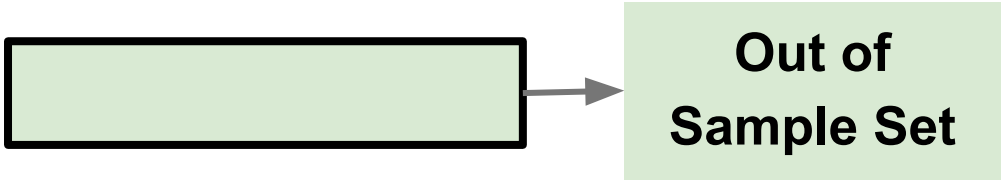
- Each feature gets a weight
- Sign of the weight
  - Positive → increases predicted probability
  - Negative → decreases predicted probability
- Size of the weight
  - Larger absolute value → stronger effect
- Linear in log-odds (logit)
  - Easy to explain how each feature affects the prediction

How do we find the weights?

Training a Machine Learning Model

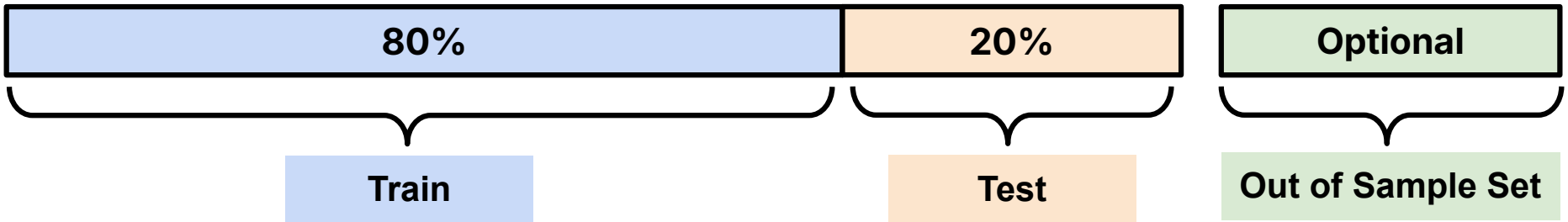


Optional:



Training a Machine Learning Model

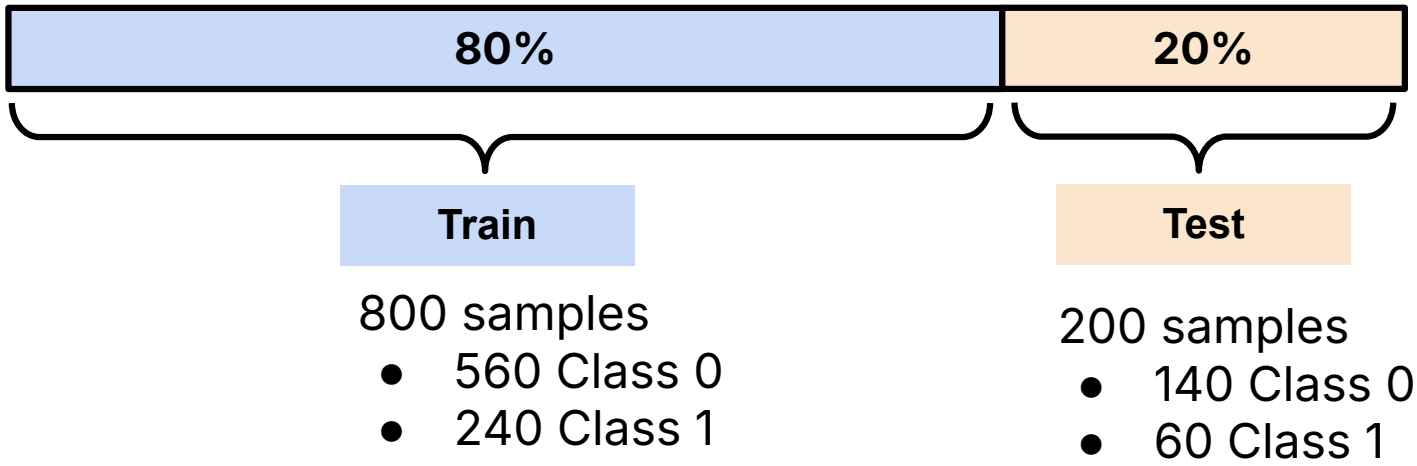
Split	Typical %	Role
Train	80%	Model sees this; Used to learn patterns from the data
Test	20%	Used to tune the model during development.
Out of Sample	N/A	If available: completely new data; touched only once at the very end



Training a Machine Learning Model: Stratify Sets

Stratified splitting is a method of dividing data into training and testing sets so that each set has approximately the same class proportions as the full dataset.

For our example:



Training a Machine Learning Model: Data Leakage

Data leakage happens when information from the test set is used during training or preprocessing, causing the model evaluation to be overly optimistic.

- Scaling before splitting
  - Fitting a scaler on the full dataset before creating train/test sets
- Feature selection using the full dataset
  - Choosing features based on all data, including the test set
- Using future information
  - In time-series problems, using data from the future to predict the past
- Target leakage
  - Including a variable that directly reveals the answer, such as a diagnosis code that only appears after the outcome is known

Training a Machine Learning Model: Feature Scaling

Feature scaling makes different variables comparable by putting them on the same scale.

- Large-scale features can dominate the model
  - Example: Gene expression level and patient age
- Scaling puts features on a comparable range
  - This helps the model treat them more fairly
- Important for logistic regression
  - Makes optimization more stable and coefficients easier to compare
- Fit on training data only
  - Then apply the same scaling to the test set to avoid *data leakage*

Training a Machine Learning Model: Feature Selection

Feature selection is the process of choosing a subset of the available features to use in a model.

- Feature selection is a Goldilocks problem
  - Too few features may miss important information
  - Too many features can add noise and reduce interpretability
- Good features are informative, relevant, and not redundant
- Common ways to choose features
  - Domain knowledge
  - Statistical tests or correlation
  - Model performance

Training a Machine Learning Model: Feature Selection

How many features should be in my model?

- The right number depends on the problem
  - Dataset size
  - Noise level
  - Number of relevant variables
  - Need for interpretability

Classification: Model Evaluation

How well does the model perform on unseen test data?

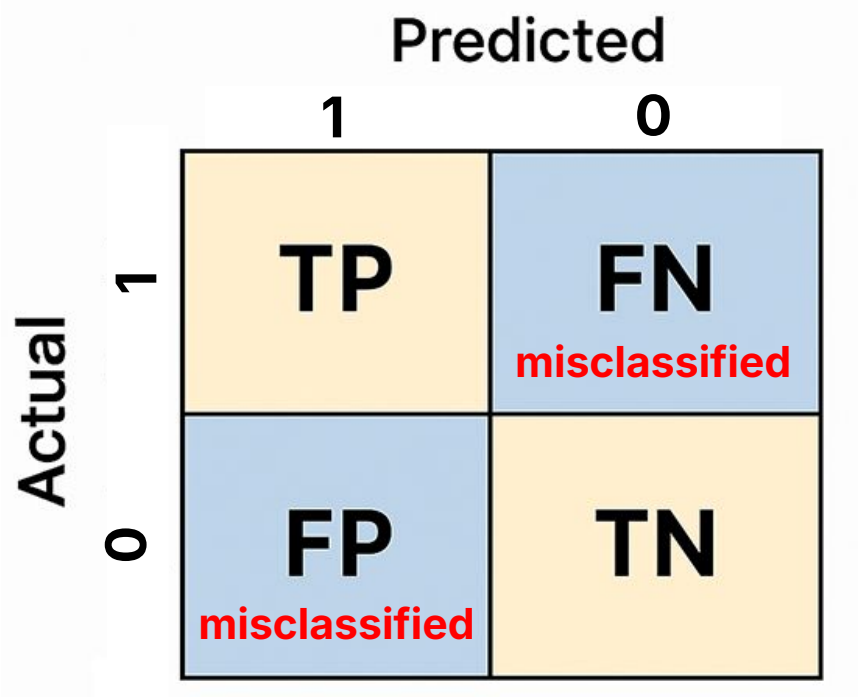
Options:

- Confusion matrix
- Accuracy
- Sensitivity
- Specificity
- Precision
- Recall
- Receiver Operating Characteristic Curve
- Precision-Recall Curve

Definition: Confusion Matrix

A **confusion matrix** is a table that compares the model's predicted classes with the true classes.

- ✓ **TP: True positive**
  - 1 predicted as 1
- ✗ **FN: False negative**
  - 1 predicted as 0
- ✓ **TN: True negative**
  - 0 predicted as 0
- ✗ **FP: False positive**
  - 0 predicted as 1



Definitions: Accuracy, Precision, Recall

Accuracy: the proportion of all predictions that are correct.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Sensitivity: the proportion of *actual positives* that are correctly identified. Also known as **recall.**

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

Specificity: the proportion of *actual negatives* that are correctly identified.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Precision: the proportion of *predicted positives* that are actually positive. Also known as **positive predictive value.**

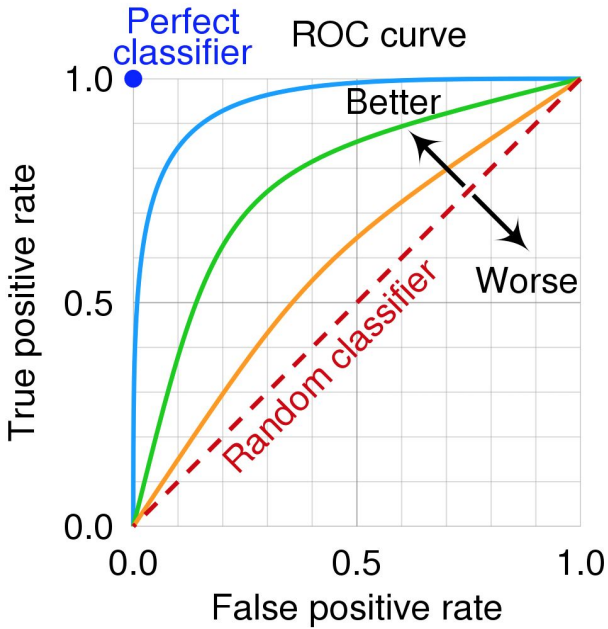
$$\text{Precision} = \frac{TP}{TP + FP}$$

Note: all these metrics are evaluated at **one** threshold!

Definitions: Receiver Operating Characteristic Curve

The **receiver operating characteristic (ROC) curve** is a plot of true positive rate (TPR) vs. false positive rate (FPR) across all possible classification thresholds.

- TPR = Sensitivity, Recall, Detection Rate
- FPR = 1 − Specificity, False Alarm Rate
- A curve closer to the upper-left corner indicates better performance
- ROC Area Under Curve (ROC AUC): summarizes the ROC curve into a single number



Example 1: Classification Pipeline

For our dataset, we will:

- We will use an 80/20 training/testing split
- We will standardize the training data using the mean and standard deviation
- We will apply the same scaling parameters to the testing data
- Train logistic regression models using all combinations of 1 to 5 features
  - 31 total feature combinations
- Compare models using the same evaluation metrics

Example 1: Initial Results

# of Features	Features	Training ROC AUC	Testing ROC AUC
5	Feature_1 + Feature_2 + Feature_3 + Feature_4 + Feature_5	0.994	**1**
4	Feature_2 + Feature_3 + Feature_4 + Feature_5	0.993	**1**
4	Feature_1 + Feature_2 + Feature_4 + Feature_5	0.994	**1**
4	Feature_1 + Feature_2 + Feature_3 + Feature_5	0.994	**1**
2	**Feature_1 + Feature_2**	0.994	**1**
3	Feature_2 + Feature_4 + Feature_5	0.993	**1**
3	Feature_2 + Feature_3 + Feature_5	0.993	**1**
3	Feature_1 + Feature_2 + Feature_5	0.994	**1**
3	Feature_1 + Feature_2 + Feature_4	0.994	**1**
2	Feature_2 + Feature_5	0.993	**1**

Traditional Machine Learning

SIAM LS Minitutorial #1

https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • **Preventing Overfitting** • Ex #2: Application
50 Mins	Deep Learning
	• Definitions • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Definition: Validation

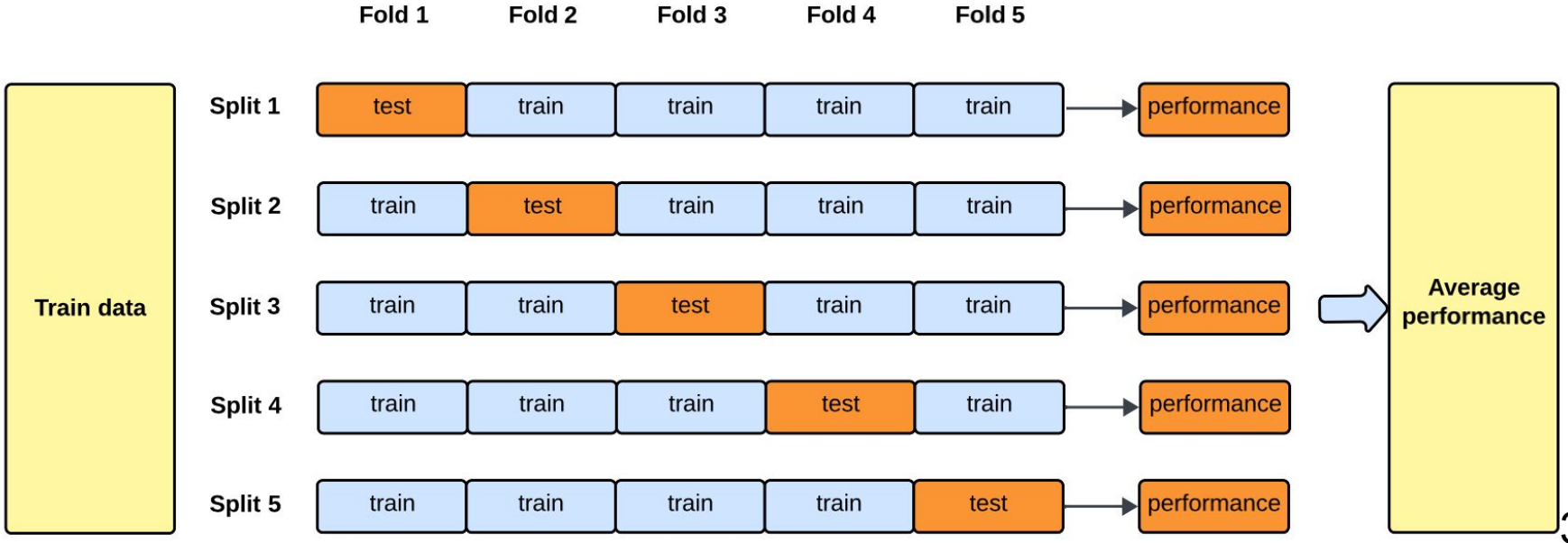
Validation in machine learning is the process of evaluating models on a separate dataset used during development to help choose features, tune hyperparameters, and compare models.

Cross-validation (CV) is a statistical technique used to evaluate how well a machine learning model generalizes to unseen data.

- Used to check model performance while building the model, before the final test evaluation.
- It is typically used to choose:
  - model type
  - feature selection
  - hyperparameters
  - decision threshold

Definition: k-Fold Cross-Validation

K-fold cross-validation is a method for evaluating a model by splitting the training data into *k* parts, training on *k-1* parts, and validating on the remaining part. This is repeated *k* times so each part is used once as validation.



Example 1: Classification Pipeline

For our dataset, we will:

- Use an 80/20 training/testing split
- Standardize the training data using the mean and standard deviation
- Apply the same scaling parameters to the testing data
- Train logistic regression models using all combinations of 1 to 5 features
  - 31 total feature combinations
- **Use 5-fold cross-validation**
  - Each model will be trained on 4 folds and validated on the remaining fold
- **Performance will be reported as the average across all 5 folds**

Example 1: Results after CV

# of Features	Features	Mean CV ROC AUC	Std CV ROC AUC
3	Feature_1 + Feature_2 + Feature_5	0.995	0.004
4	Feature_1 + Feature_2 + Feature_3 + Feature_5	0.995	0.004
4	Feature_1 + Feature_2 + Feature_4 + Feature_5	0.995	0.005
2	**Feature_1 + Feature_2**	0.995	0.004
3	Feature_1 + Feature_2 + Feature_4	0.995	0.004
5	Feature_1 + Feature_2 + Feature_3 + Feature_4 + Feature_5	0.995	0.004
3	Feature_1 + Feature_2 + Feature_3	0.995	0.004
4	Feature_1 + Feature_2 + Feature_3 + Feature_4	0.995	0.004
1	Feature_2	0.994	0.005
2	Feature_2 + Feature_5	0.994	0.005

Penalizing Unnecessary Features During Model Selection

Akaike information criterion (AIC) is a measure used to compare models by balancing goodness of fit and model complexity. Lower AIC values indicate a better model.

$$\mathrm{AIC} = 2d - 2\ell$$

$$\ell = \sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

d = number of parameters
ℓ = maximized log-likelihood

Bayesian information criterion (BIC) is a measure used to compare models by balancing goodness of fit and model complexity, with a stronger penalty for extra parameters than AIC. Lower BIC values indicate a better model.

$$\mathrm{BIC} = d \ln(n) - 2\ell$$

$$\ell = \sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

d = number of parameters
n = number of observations

Example 1: Using AIC/BIC

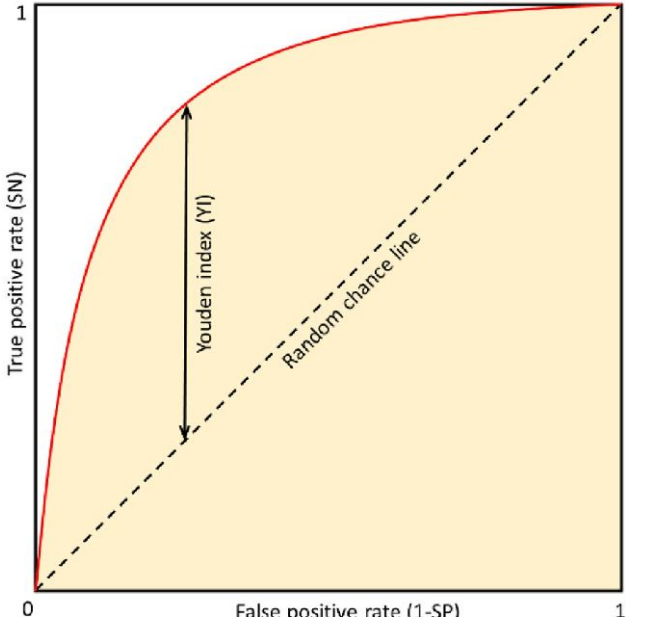
# of Features	Features	Log-Likelihood	AIC	BIC
3	Feature_1 + Feature_2 + Feature_5	-55.94	119.88	139.511
2	Feature_1 + Feature_2	-57.043	120.087	134.81
4	Feature_1 + Feature_2 + Feature_4 + Feature_5	-55.859	121.719	146.257
4	Feature_1 + Feature_2 + Feature_3 + Feature_5	-55.912	121.825	146.364
3	Feature_1 + Feature_2 + Feature_3	-56.999	121.998	141.629
3	Feature_1 + Feature_2 + Feature_4	-57.003	122.007	141.638
5	Feature_1 + Feature_2 + Feature_3 + Feature_4 + Feature_5	-55.83	123.659	153.106
4	Feature_1 + Feature_2 + Feature_3 + Feature_4	-56.957	123.914	148.453
1	Feature_2	-65.734	135.467	145.283
2	Feature_2 + Feature_5	-65.478	136.956	151.679

ML: Picking the Optimal Threshold - Youden's J Statistic

- Summarizes overall classification performance

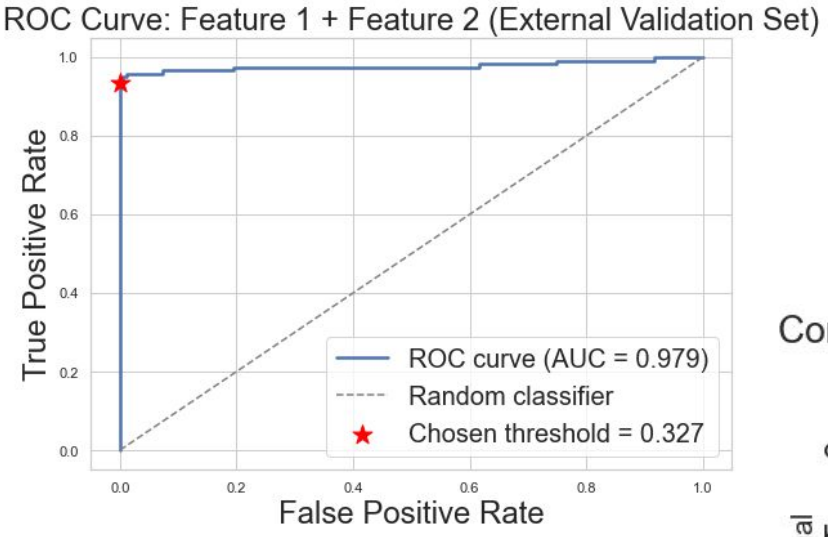
  J = Sensitivity + Specificity - 1

- Ranges from 0 (no better than chance) to 1 (perfect classification)
- The optimal threshold is the one that maximizes J
- Balances:
  - Sensitivity: correctly identifying positives
  - Specificity: correctly identifying negatives
- Commonly used to choose biomarker or diagnostic cutoffs



Example 1: Application to Out-of-Sample Data

Model Built using Feature 1 + Feature 2



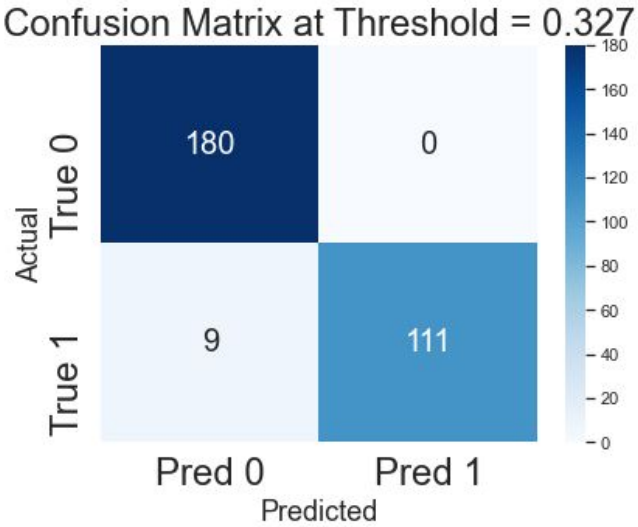
Accuracy: 97.0%

Precision: 1.000

Recall: 0.925

Sensitivity: 0.925

Specificity: 1.000



Traditional Machine Learning

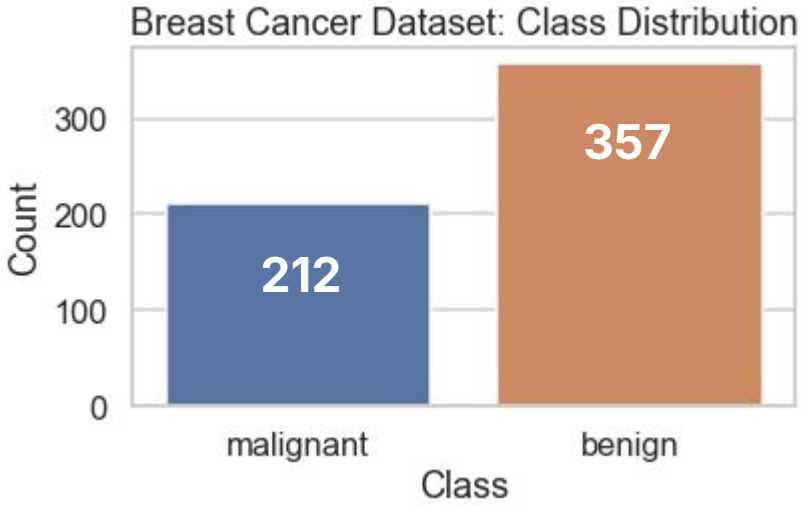
SIAM LS Minitutorial #1

https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • Preventing Overfitting • **Ex #2: Application**
50 Mins	Deep Learning
	• Definitions • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Example 2: Breast Cancer Classification

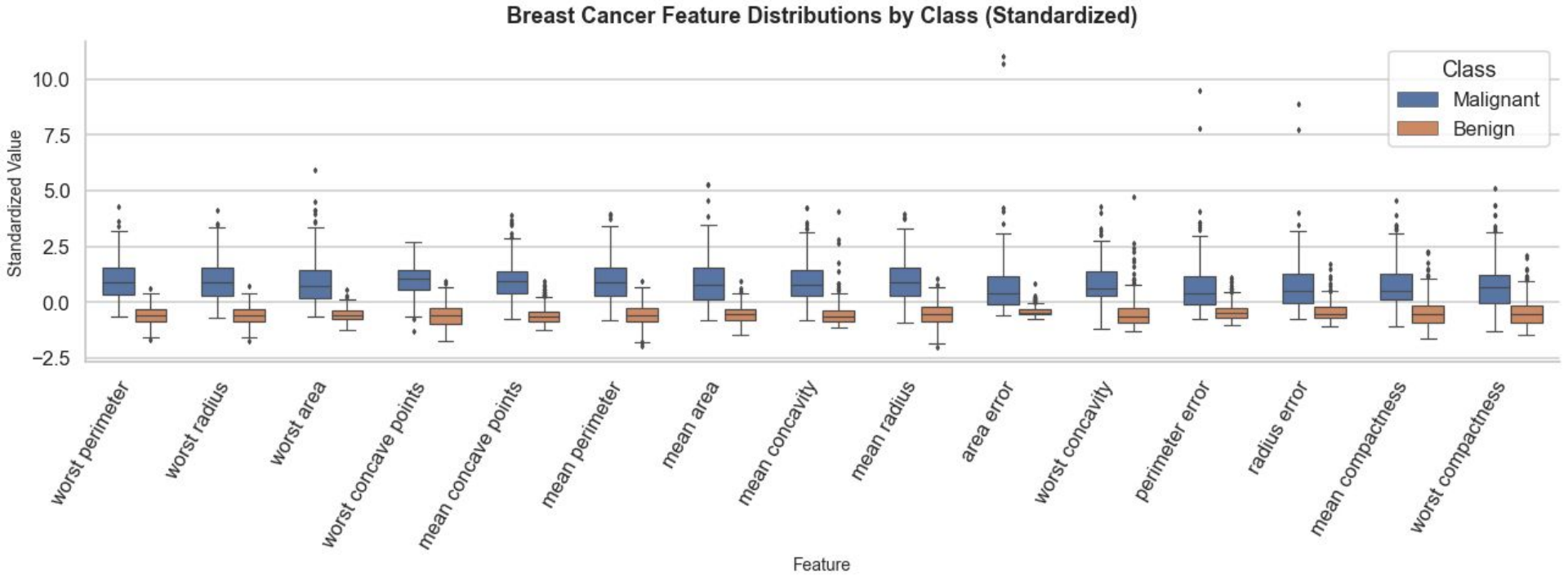
- **Source:** scikit-learn's built-in copy of the Breast Cancer Wisconsin (Diagnostic) dataset.
- **Task:** Binary classification — predict whether a tumor is malignant or benign.
- **Size:** 569 samples and 30 numeric features.
- **Feature meaning:** Features are computed from digitized images of fine needle aspirates (FNA) of breast masses and describe characteristics of the cell nuclei.



Python notebook: ML_Example2_BreastCancer.ipynb

Example 2: Breast Cancer Classification

Not as easy to identify the important features!



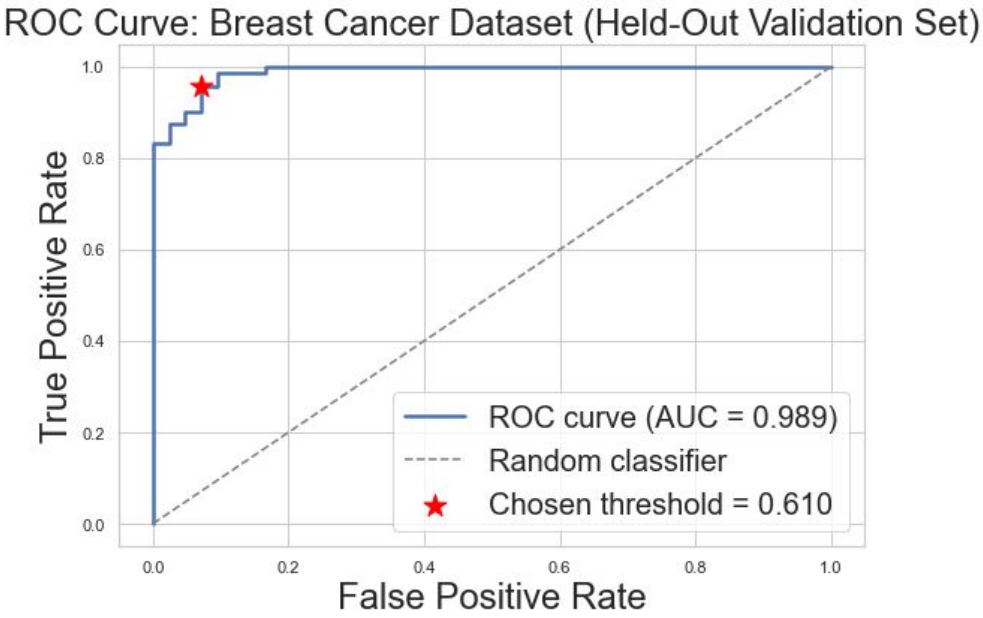
Example 2: Breast Cancer Classification

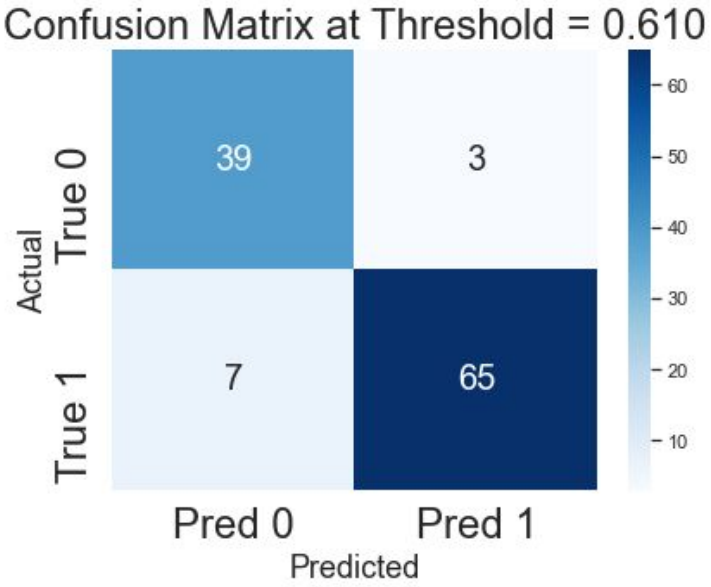
Num Features	Features	Mean CV ROC AUC	Std CV ROC AUC	AIC	BIC
3	mean concave points + worst radius + worst texture	0.994	0.005	76.186	91.513
3	worst texture + worst area + worst concave points	0.994	0.01	76.913	92.24
3	mean concave points + worst texture + worst area	0.994	0.005	77.038	92.365
3	mean texture + worst radius + worst smoothness	0.994	0.006	79.494	94.821
3	area error + worst texture + worst concave points	0.994	0.004	80.596	95.923

Example 2: Breast Cancer Classification

mean concave points + worst radius + worst texture

Accuracy: 91.2%
Precision: 0.956
Recall: 0.903
Sensitivity: 0.903
Specificity: 0.929





Deep Learning

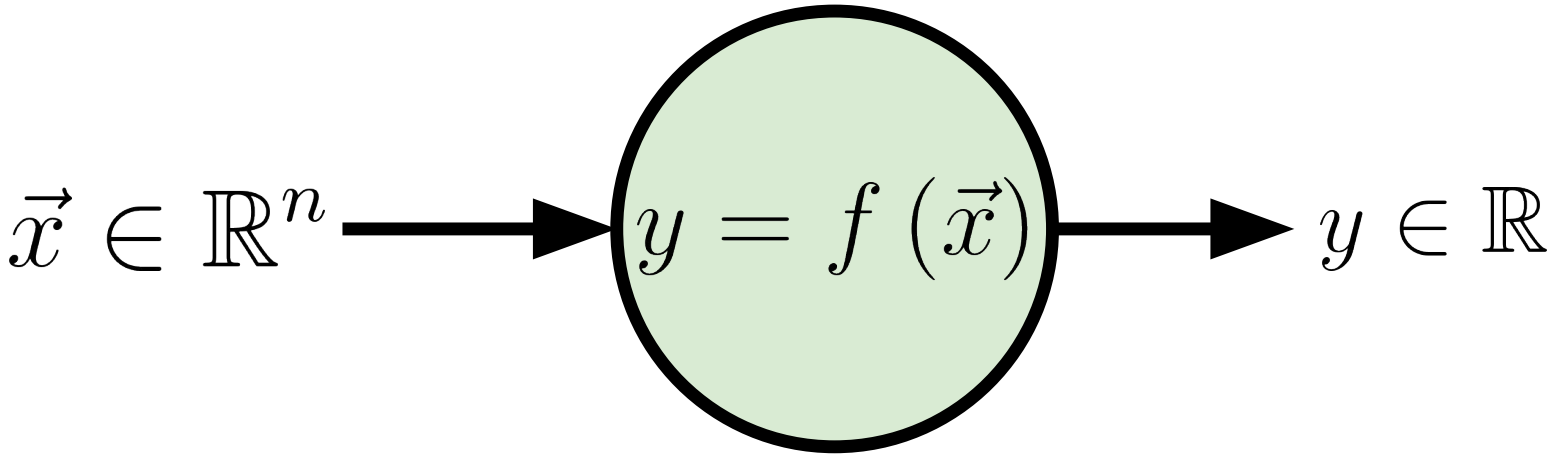
SIAM LS Minitutorial #1

https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • Preventing Overfitting • Ex #2: Application
50 Mins	Deep Learning
	• **Definitions** • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Definition: Neuron

Artificial **neurons** are modeled on biological neurons. They receive inputs, transform them and transmit outputs



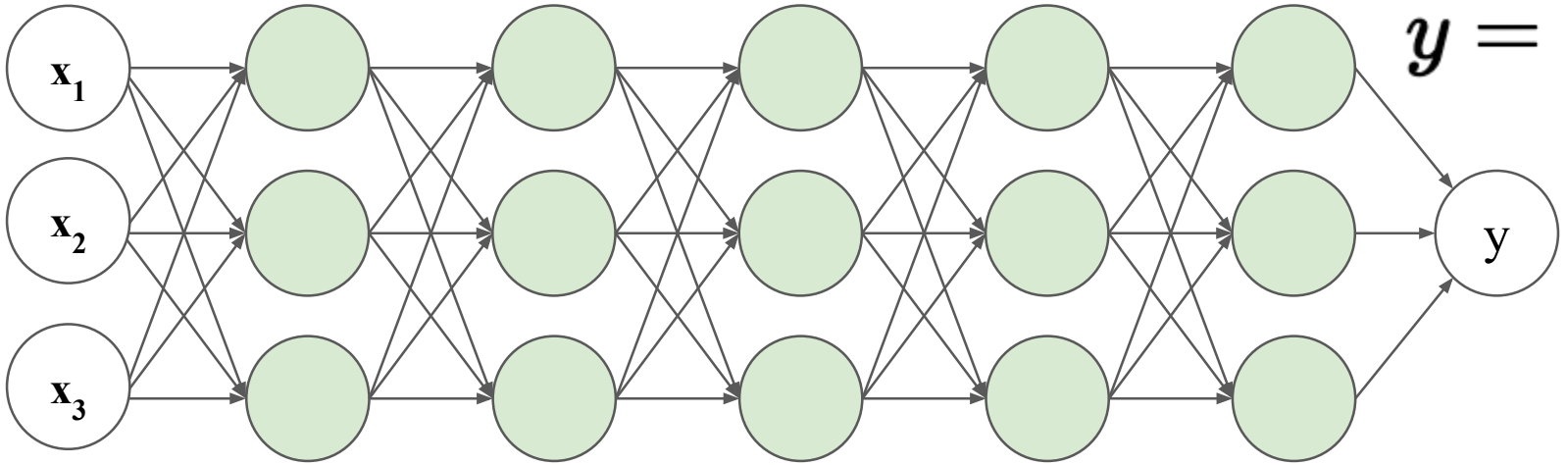
Definition: Neural Network

A **neural network (NN)** is a graph of neurons. The nodes are neurons and the edges are connections between them.

$\vec{x} \in \mathbb{R}^3$

The NN is of course a function.

$y = f(\vec{x})$



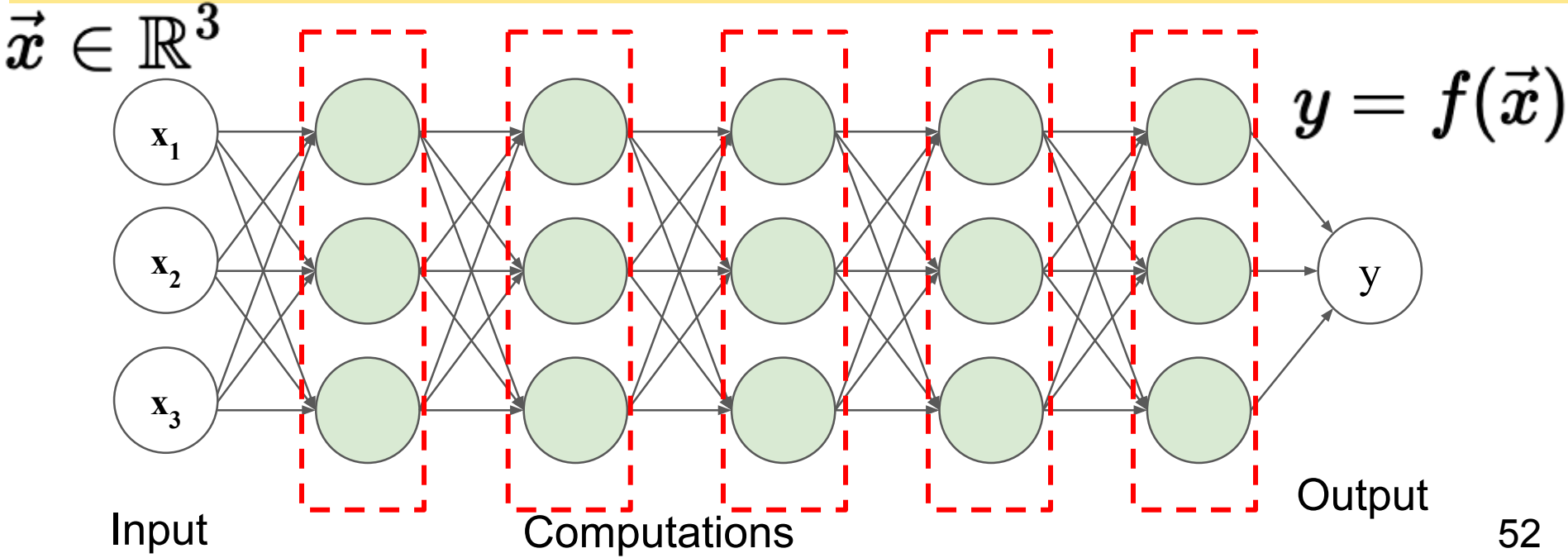
Input

Computations

Output

Definition: Layer

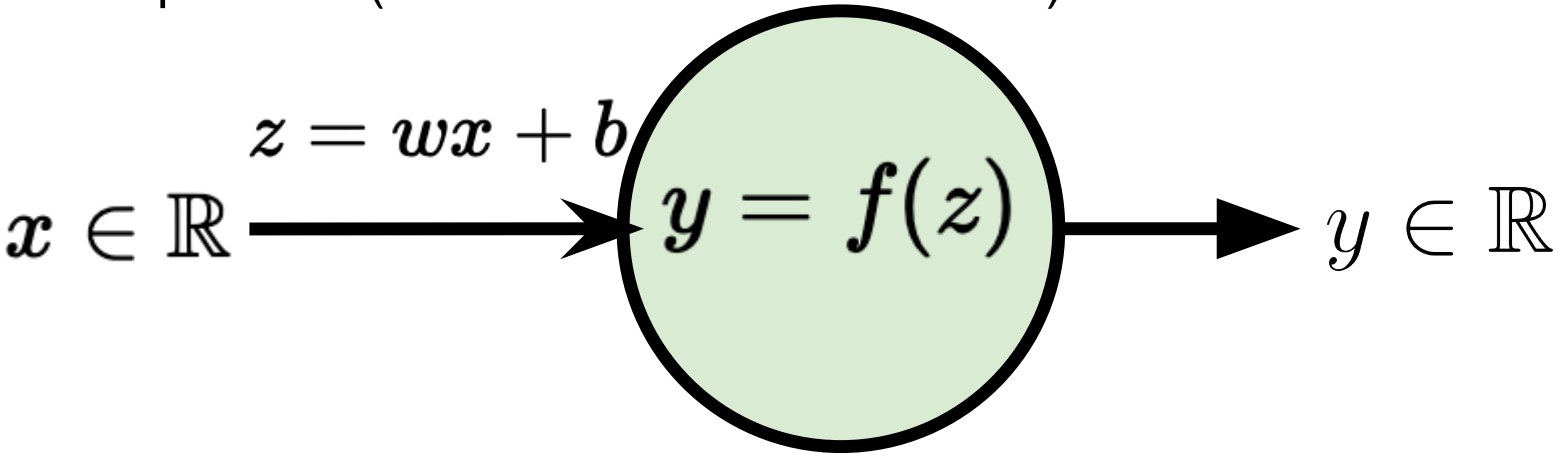
Many common NNs consist of a series of layers from input layer (left) to the output layer (right). A **layer** of a neural network is a set of nodes that receive some input from the previous layer.



Definition: Activation Function

An **activation function** is a nonlinear function applied to a neuron's **pre-activation value z** (the weighted sum of inputs plus a bias), producing the neuron's output.

The goal is to have a behavior that is “0” when negative and “big” when positive. (Let’s look at the 1-D version)



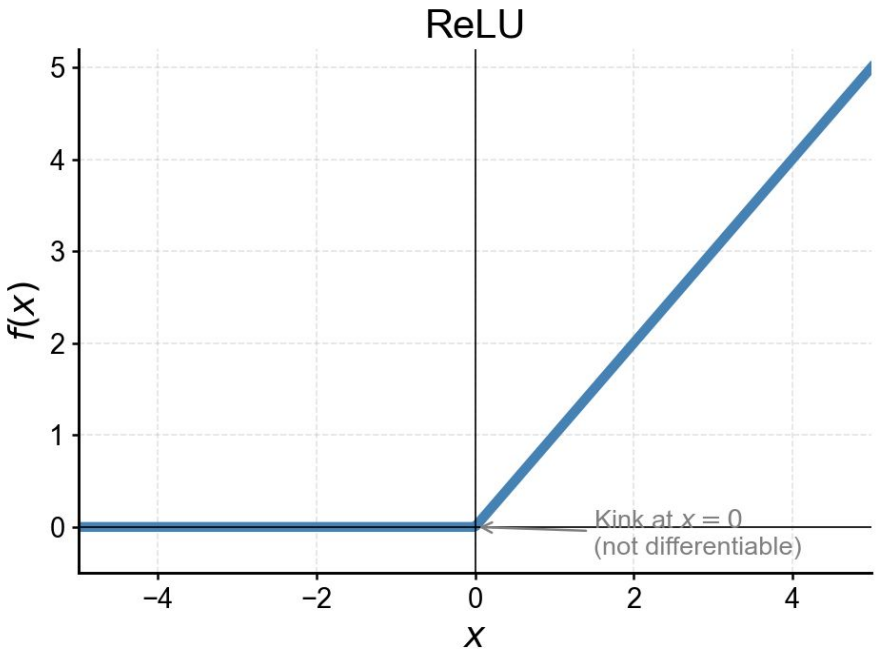
Common Activation Functions (1 →1)

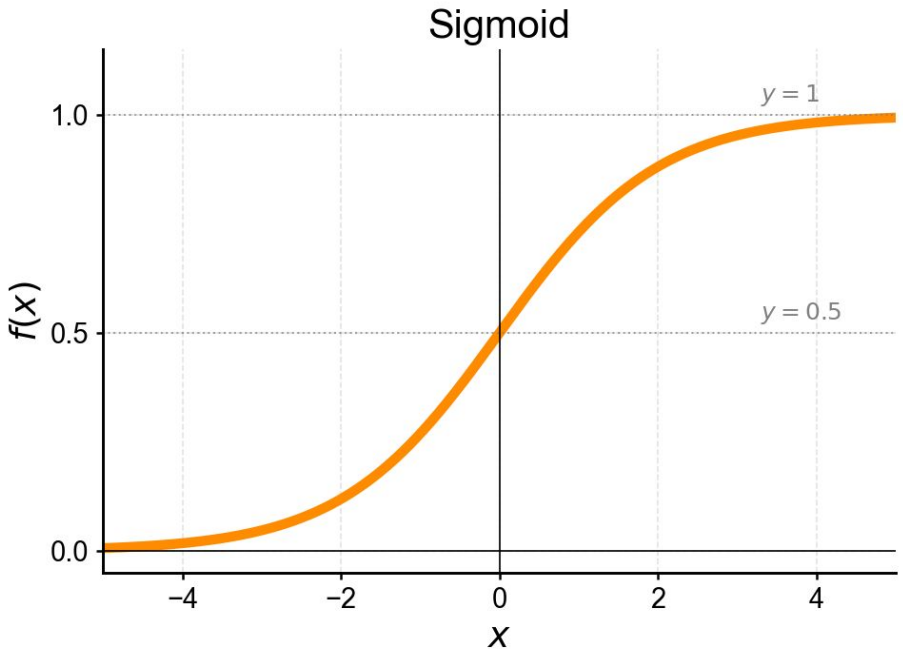
ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

Sigmoid

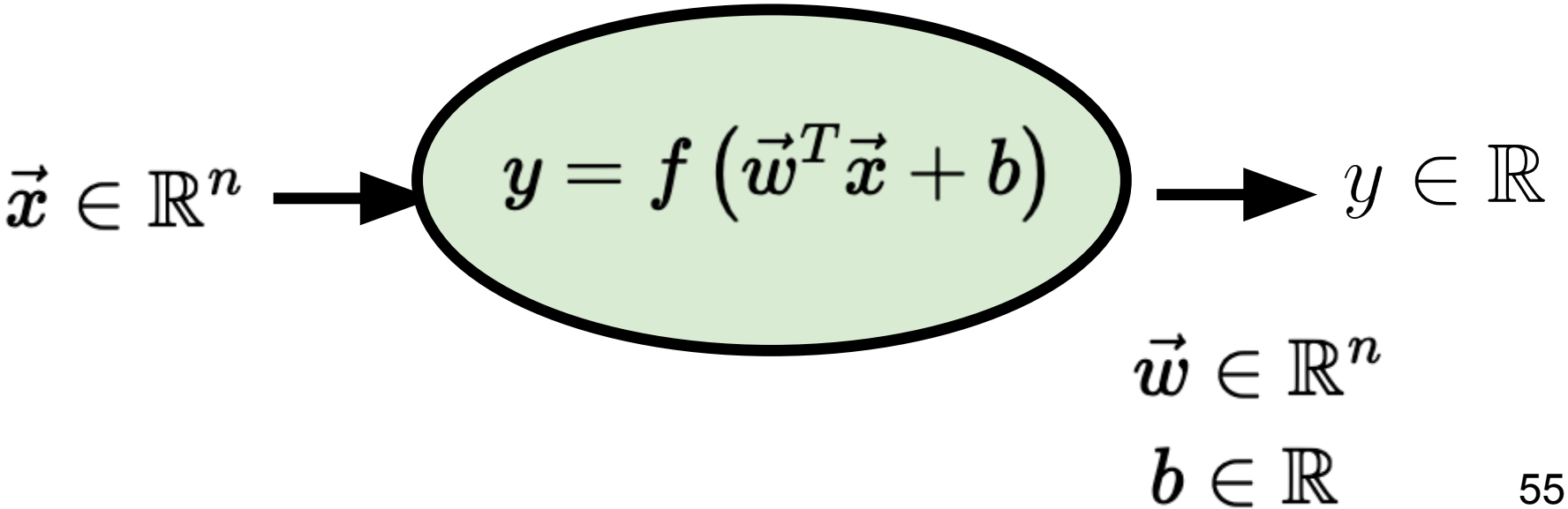
$$f(x) = \frac{1}{1 + e^{-x}}$$





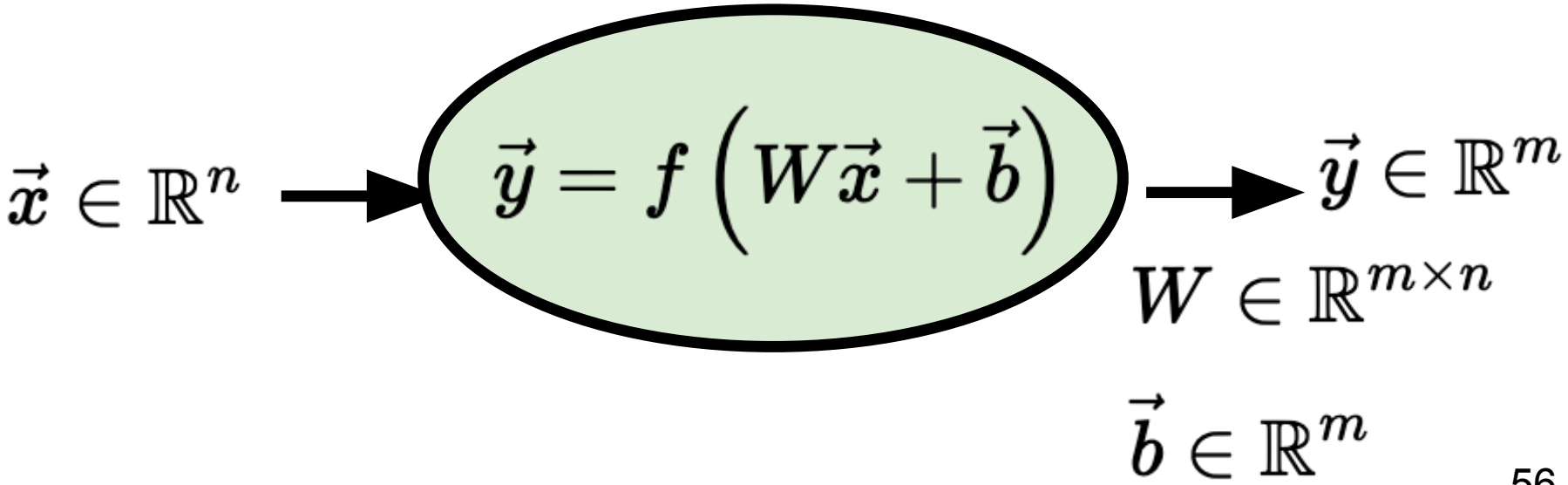
Definitions: Weights and Biases

The learnable parameters of a neuron are the **weights** ($\vec{w}$) and **bias** (b) that define the pre-activation value: $z = \vec{w}^T \vec{x} + b$.



Definitions: Weights and Biases

In general neurons are able to transform any input dimension $\mathbf{n}$ to any output dimension $\mathbf{m}$.



Training Neural Networks

Weights and bias are the learnable parameters of the neural network. The formula for the final output layer is (generally) not explicit.

Training is finding weights and biases that make the output match the data as well as possible.

$$y = f\left(\vec{w}^T \vec{x} + b\right)$$

Given: Data $\{(\vec{x}_i, y_i)\}_{i=1}^N$ where $\vec{x}_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$

Find: $\vec{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$

Where: $y_i \approx f\left(\vec{w}^T \vec{x}_i + b\right)$

Definition: Loss Function

The **loss function** is what we minimize to identify weights and biases. Generally the average per term loss.

Given: Data $\{(\vec{x}_i, y_i)\}_{i=1}^{N}$ where $\vec{x}_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$

Find: $\vec{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$

Where: $y_i \approx f\left(\vec{w}^T \vec{x}_i + b\right)$

$$L(\vec{w}, b) = \frac{1}{N} \sum_{i=1}^{N} \ell\left(y_i,\ f(\vec{w}^T \vec{x}_i + b)\right)$$

The **per term loss function** ℓ depends on your loss function depends on problem context and (for more than 1 node) f is not explicit.

Special Case: Binary Classification & Sigmoid Activation

The **loss function** is what we minimize to identify weights and biases.

Given: Data $\{(\vec{x}_i, y_i)\}_{i=1}^{N}$ where $\vec{x}_i \in \mathbb{R}^n$ and $y_i \in \{0, 1\}$

Find: $\vec{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$

Where: $y_i \approx f(\vec{w}^T \vec{x}_i + b) = \left(1 + e^{-(\vec{w}^T \vec{x}_i + b)}\right)^{-1}$

$$L(\vec{w}, b) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log\left(1 + e^{\vec{w}^T \vec{x}_i + b}\right) + (1 - y_i) \log\left(1 + e^{-(\vec{w}^T \vec{x}_i + b)}\right) \right]$$

Special Case: Binary Classification & Sigmoid Activation

A single neuron with **sigmoid activation** and **binary cross-entropy loss** is logistic regression

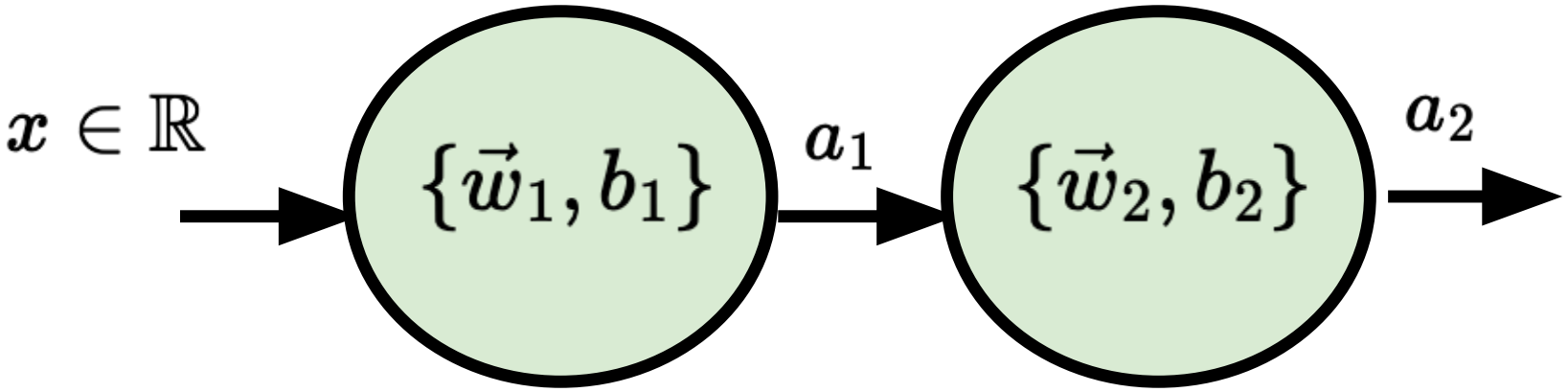
$$p_i = \text{Prob}(y_i = 1|\vec{x}_i) = \frac{e^{(\vec{w}^T\vec{x}_i+b)}}{1+e^{(\vec{w}^T\vec{x}_i+b)}}$$

$$L(p_i) = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]$$

Neural networks generalize this by stacking many such neurons - the single neuron is the base case.

Q: What about when our loss doesn't have an explicit form??

Special Case: Two Neurons (1-1)



$$a_1 = f_1(w_1 x + b_1) \qquad a_2 = f_2(w_2 \cdot a_1 + b_2)$$

$$a_2 = f_2(w_2 \cdot f_1(w_1 x + b_1) + b_2)$$

Special Case: Two Neurons (1-1)

$$a_2 = f_2(w_2 \cdot f_1(w_1 x + b_1) + b_2)$$

The per-term loss is $\ell(y,a_2)$ and we need to take derivatives of L with respect to parameters w_1, w_2, b_1, b_2.

We just use our friend **chain rule** (each term is easily computable)

$$\frac{\partial \ell(y,\ f_2(w_2 \cdot f_1(w_1 x + b_1) + b_2))}{\partial w_1} = \frac{\partial \ell}{\partial f_2} \cdot \frac{\partial f_2}{\partial f_1} \cdot \frac{\partial f_1}{\partial w_1}$$

We have gradients and can locally move in a good direction for each weight and bias!

Definition: Gradient Descent (GD)

The **gradient descent** algorithm lets us update all weights and biases. Each node only needs to know it's *forward* computation and *derivative*

Algorithm:

1) Forward pass: Compute the loss over all N samples.
2) Backward pass: Apply the chain rule through the network to compute each derivative
3) Update parameters: Take a small step in the direction that reduces loss (negative gradient).

Repeat (1) - (3) until convergence.

Definition: *Stochastic* Gradient Descent (SGD)

The **stochastic gradient descent** algorithm lets us update all weights and biases.

Algorithm:

1) Forward pass: Compute the loss over *1 randomly chosen sample*
2) Backward pass: Apply the chain rule through the network to compute each derivative
3) Update parameters: Take a small step in the direction that reduces loss (negative gradient).

Repeat (1) - (3) until convergence (i.e., loss stops decreasing) or early stopping.

Definition: *Minibatch Stochastic* Gradient Descent

The **minibatch stochastic gradient descent** algorithm lets us update all weights and biases.

Algorithm:

1) Forward pass: Compute the loss over *a random subset of B samples*
2) Backward pass: Apply the chain rule through the network to compute each derivative
3) Update parameters: Take a small step in the direction that reduces loss (negative gradient).

Repeat (1) - (3) until convergence (i.e., loss stops decreasing) or early stopping.

Definition: Learning Rate

The **learning rate** controls how large a step we take when updating parameters

- Too Large: Unstable
- Too Small: Training slow or gets stuck

We will use Adam, an adaptive optimizer, that updates the learning rate for each parameter by normalizing each parameter update by its historical gradient variance.

- Adam = **Ad**aptive **M**oment estimation.
- All algorithms (GD, SGD, minibatch SGD) are first order methods.
- Second order methods are not generally tractable for these systems.

Definition: Batch Size and Epoch

The **batch size** B is the number of samples used to estimate the loss in each update step.

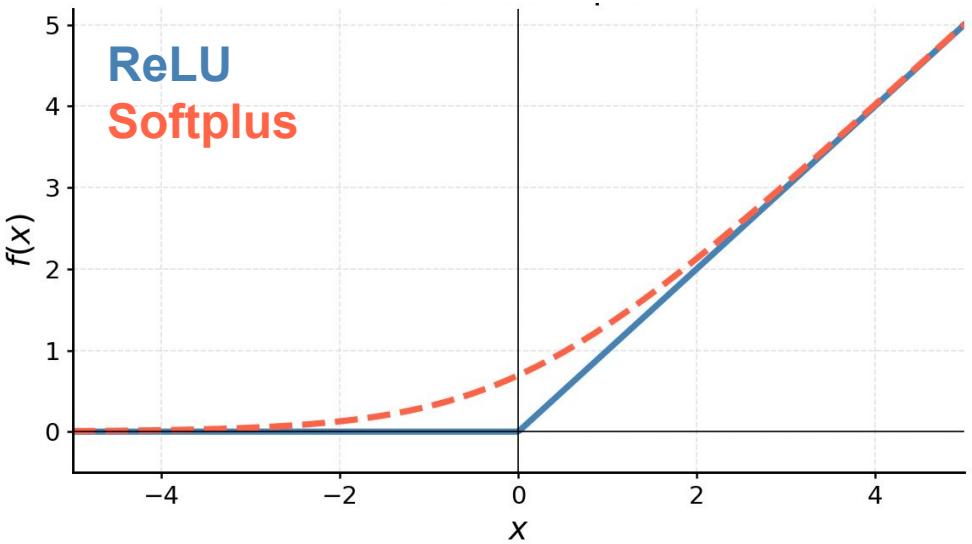
An **epoch** is a full pass through N samples. If your batch size is B, then a single epoch corresponds to N/B update steps through your chosen algorithm.

- Each epoch shuffles the training data and passes through all N samples exactly once - every sample appears in exactly one batch per epoch.
- The stochastic part refers to the “random ordering at each epoch”

Technical Challenge: ReLU derivative at 0

The derivative of ReLU doesn't exist at 0! Is deep-learning broken?

- **Subgradients** developed for convex optimization tell us we can select any value in [0,1] for the derivative at x = 0.
- We can also use **smooth approximations** to ReLU such as Softplus: $f(x) = \log(1 + e^x)$



Tensorflow/PyTorch use **automatic differentiation** and hardcode ReLU′(0) = 0 and apply chain rule mechanically at each node, evaluating exact derivatives numerically rather than symbolically or approximately.

Deep Learning

SIAM LS Minitutorial #1

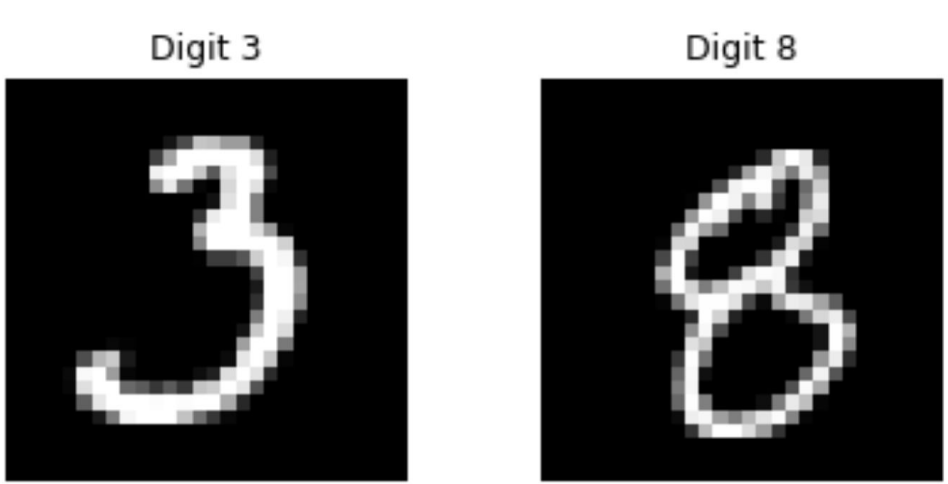
https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • Preventing Overfitting • Ex #2: Application
50 Mins	Deep Learning
	• Definitions • **Ex #1: Classification with MNIST** • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Example 1: Image Classification with MNIST

1) How do you train a neural network? (Train, Validate, Test)
2) Two different types of Neural Networks (MLP and CNN)
3) How do you get interpretable information from the NN

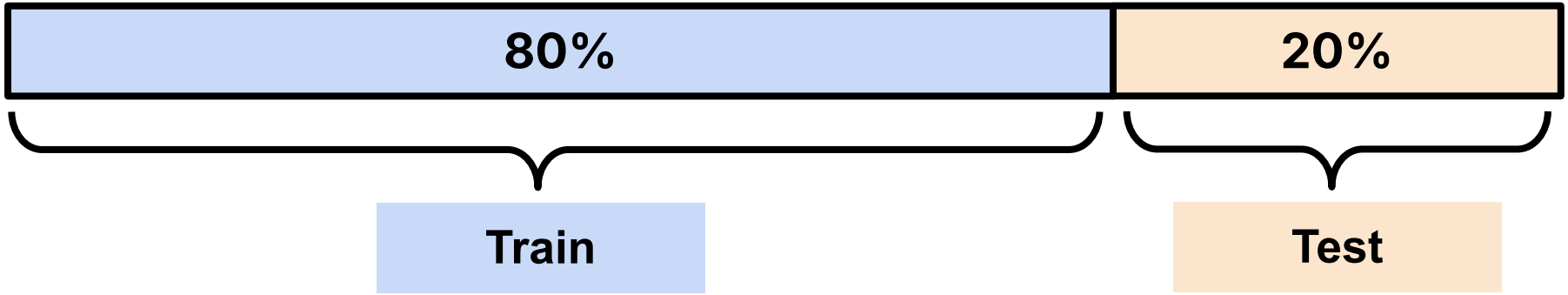
Goal: Successfully determine if a written digit is a 3 or an 8?



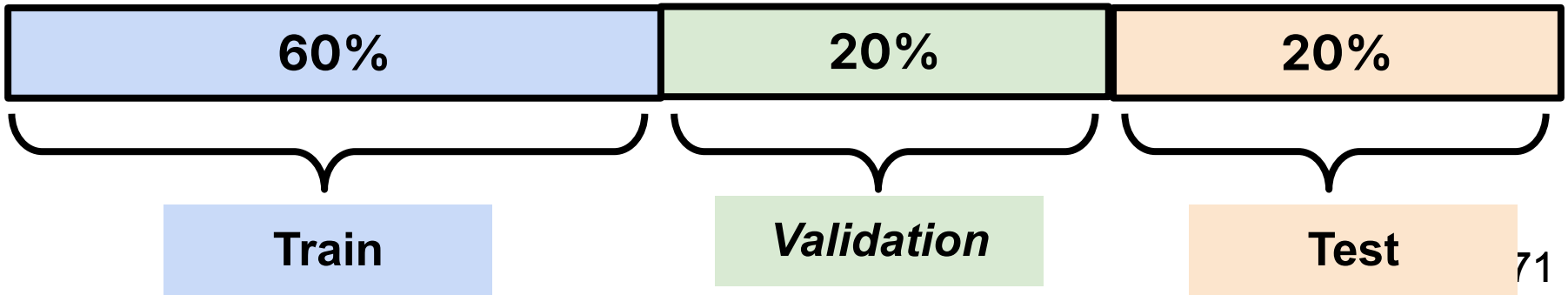
Python notebook:
DL_Example1_MNIST_to_PneumoniaMNIST.ipynb

Training a Neural Network

Traditional Machine Learning

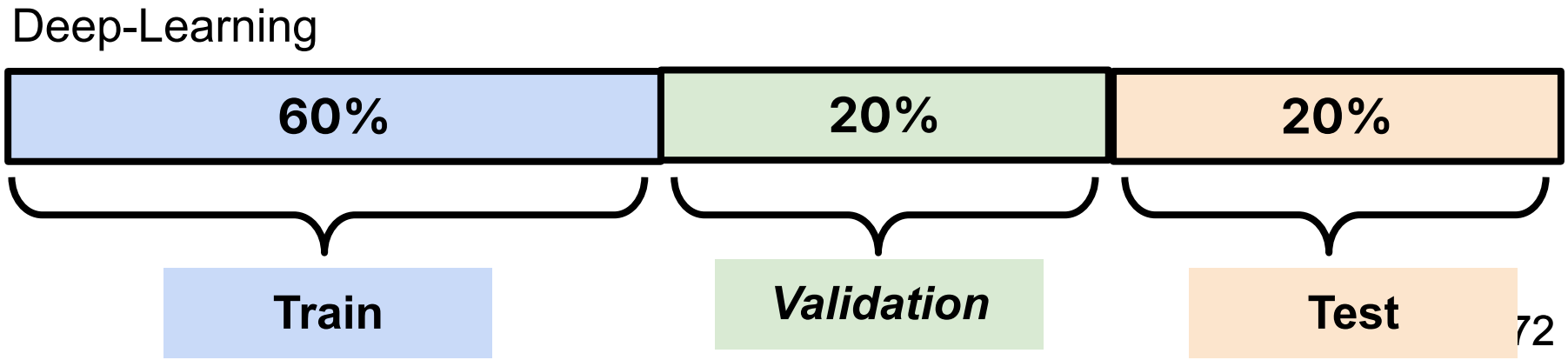


Deep-Learning



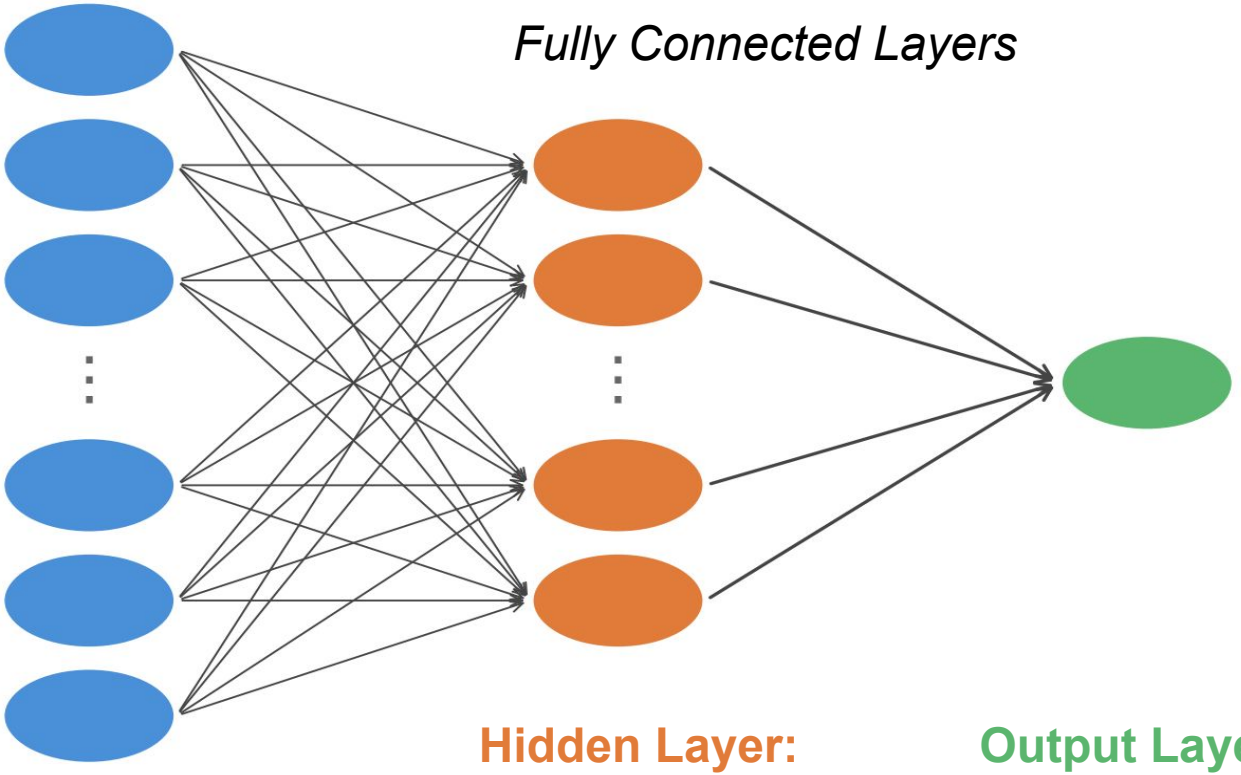
Training a Neural Network

Split	Typical %	Role
Train	60%	Model sees this; weights are updated on it
Validation	20%	Evaluated during training to monitor overfitting and tune hyperparameters
Test	20%	Held out completely; touched only once at the very end



Multi-Layer Perceptron (MLP)

28x28
Input



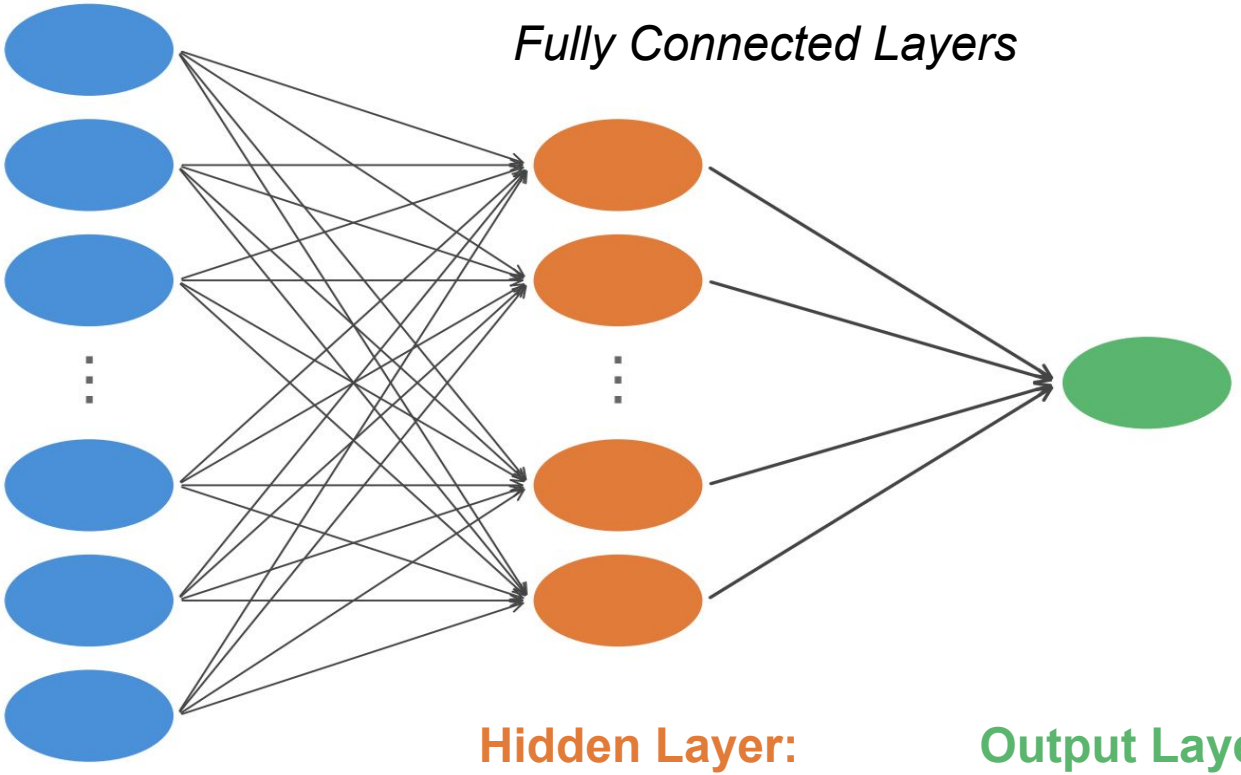
Input Layer: 784 Nodes
28 x 28 Flattened

Hidden Layer:
16 Nodes
ReLU

Output Layer:
1 Node
Sigmoid

Multi-Layer Perceptron (MLP)

# of Parameters		
	Per Node	Per Layer
Input	0	0
Hidden	785	12,560
Output	17	17
Total		**12,577**



Input Layer: 784 Nodes
28 x 28 Flattened

Hidden Layer:
16 Nodes
ReLU

Output Layer:
1 Node
Sigmoid

Multi-Layer Perceptron (MLP)

```python
def build_mlp():
    model = models.Sequential([
        layers.Flatten(input_shape=(28, 28)),
        layers.Dense(16, activation="relu"),
        layers.Dense(1,  activation="sigmoid"),
    ], name="MLP")
    model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
    return model

mlp = build_mlp()
mlp.summary()
```

- **Layers** built sequentially (as our diagram showed)
- **Loss**: Binary cross-entropy measures error on 0/1 labels
- **Optimizer**: Adam, adaptive learning rate, updates all 12,577 parameters each step

MLP Training on MNIST

Train: 8,379 | Val: 2,793 | Test: 2,794 (held out)

```
# Early stopping callback — monitors validation loss
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=25,                # stop after 25 epochs with no improvement
    restore_best_weights=True,# revert to the epoch with lowest val_loss
    verbose=1
)
# We pass this to every model.fit() call below.
```

- **Early stopping** halts training when validation loss stops improving — prevents overfitting
- `patience=25` means wait 25 epochs with no improvement before stopping; `restore_best_weights` rewinds to the best epoch

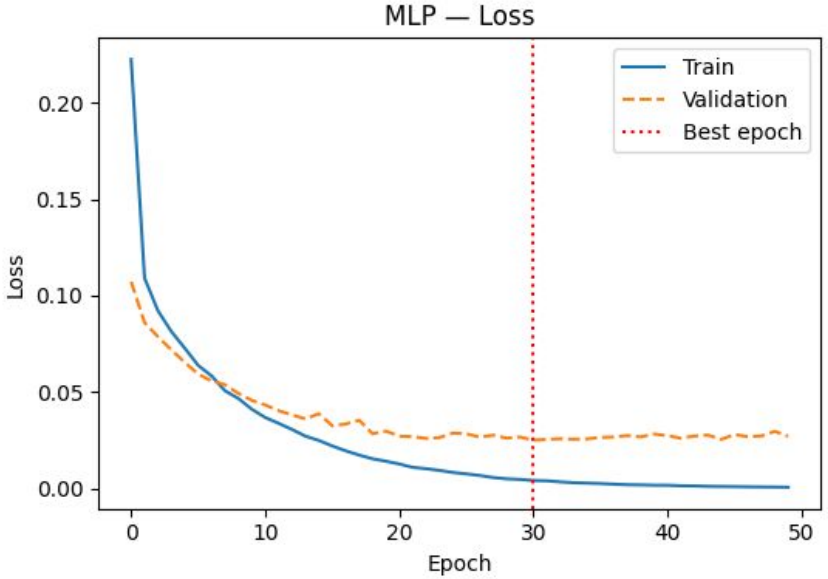
MLP Training on MNIST

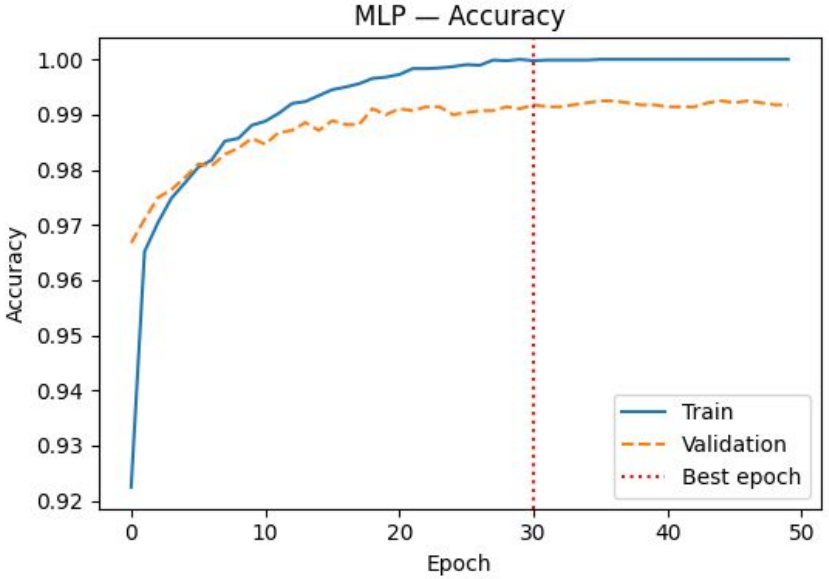
Train: 8,379 | Val: 2,793 | Test: 2,794 (held out)

```
mlp_history = mlp.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=50,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)
```

- We go for 50 epochs with a `batch_size` of 64
- We monitor validation loss (not training loss) — training loss always decreases, validation loss tells us when we're starting to overfit

Training Curves: Loss and Accuracy





- **Loss**: binary cross-entropy computed directly
- **Accuracy**: fraction correct using threshold y>0.5
- Training performance is always better than validation.
- We stop early because the validation performance stops improving.

MLP test accuracy (held-out 20%): 0.9900

MLP Summary

Pros:

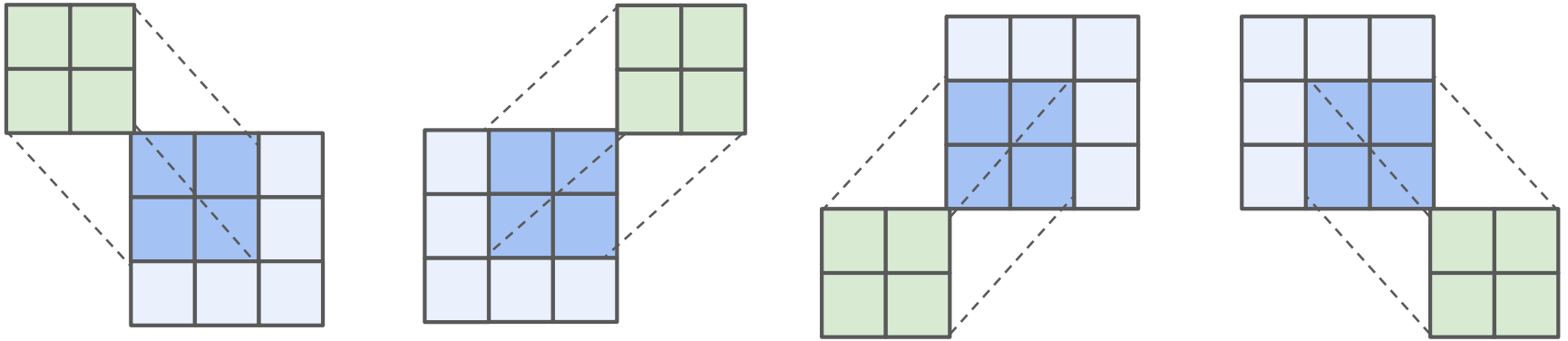
- Performed well on model training
- Early stopping helped protect against overfitting.
- Very accurate on the held out data set (99%)

Cons:

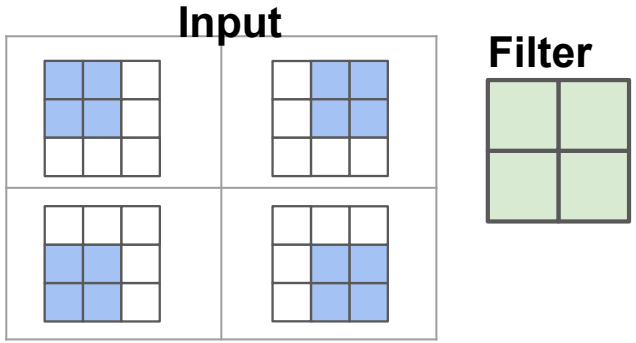
- *No spatial structure:* treats every pixel independently, ignoring that nearby pixels are related
- *Not interpretable:* no clear way to explain which parts of the image drove the prediction (black-box; garbage in garbage out)
- Needed pre-trained and annotated data (all deep learning in general)
- Likely won't generalize as well to harder datasets (e.g. medical images with subtle features)

Definition: Filter (Kernel) and Feature Map

A **filter** is a set of weights and a bias that is applied to every local patch of an input to produce a **feature map**.



We set a **patch size** slide along the image. (Here 2x2)



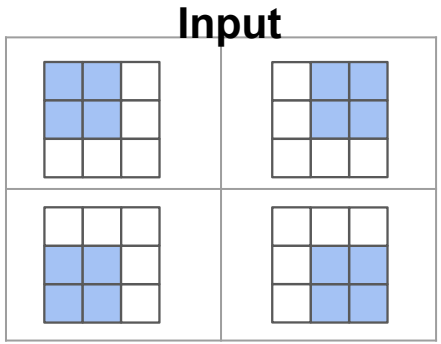
Definition: Filter (Kernel) and Feature Map

A **filter** is a set of weights and a bias that is applied to every local patch of an input to produce a **feature map**.

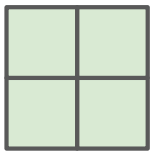
$$\begin{bmatrix} X^{(1)} & X^{(2)} \\ X^{(3)} & X^{(4)} \end{bmatrix}$$

Apply Filter

$$\begin{bmatrix} F_1 & F_2 \\ F_3 & F_4 \end{bmatrix}$$



Filter



Feature Map

Definition: Filter (Kernel) and Feature Map

A **filter** is a set of weights and a bias that is applied to every local patch of an input to produce a **feature map**.

$$\begin{bmatrix} X^{(1)} & X^{(2)} \\ X^{(3)} & X^{(4)} \end{bmatrix} \xrightarrow{\text{Apply Filter}} \begin{bmatrix} F_1 & F_2 \\ F_3 & F_4 \end{bmatrix}$$

Input (Patches)

$$X^{(k)} = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix}$$

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

$$b \in \mathbb{R}$$

Feature Map

$$F_k = f\Big(\langle W, X^{(k)} \rangle + b\Big)$$

Convolutional Neural Network (CNN)

28x28
Input

Layer1

Conv 2D
16 Filters
3x2 ReLU

Layer 2

Flatten

Layer 3

Dense
Sigmoid

Layer 4

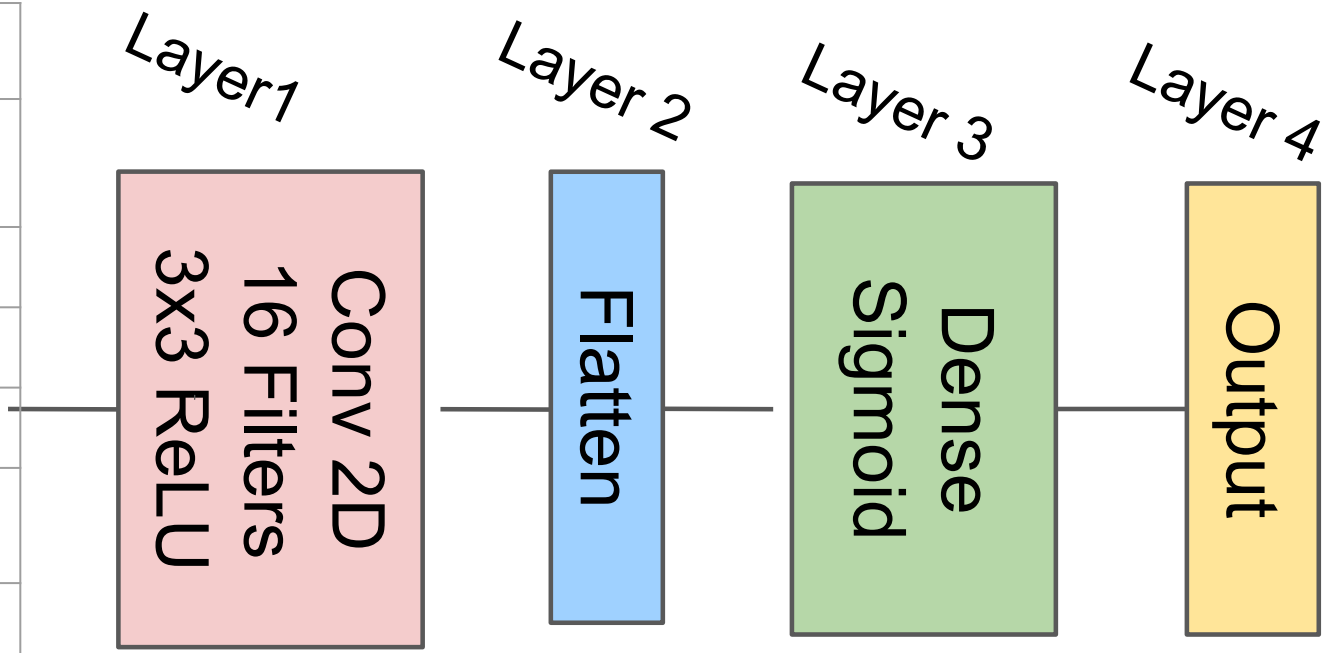
Output

Convolutional Layer:
16 Filters
ReLU

Output Layer:
1 Node
Sigmoid

Convolutional Neural Network (CNN)

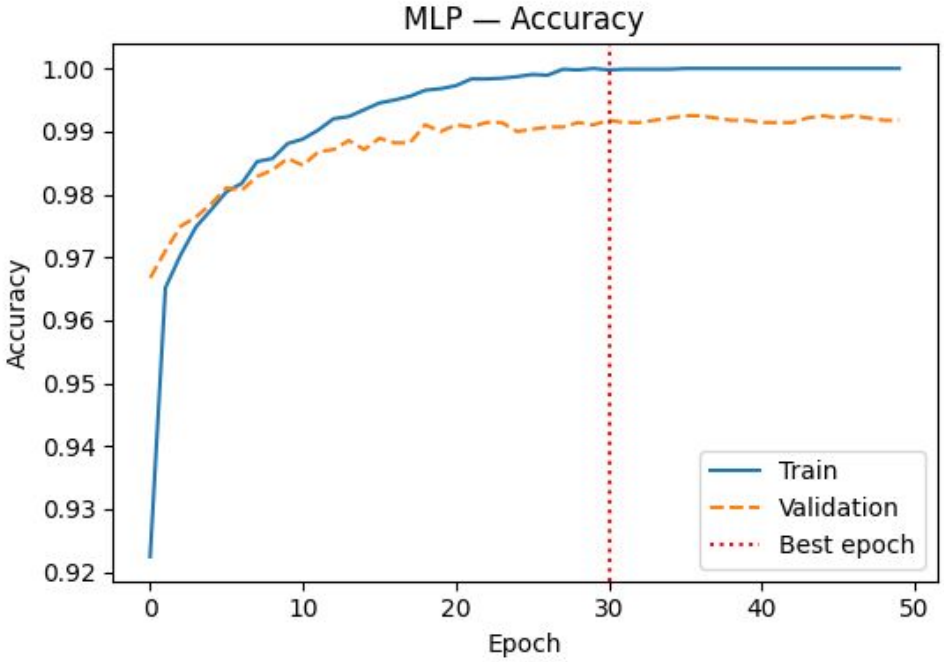
# of Parameters		
	Per Node	Per Layer
Input	0	0
CNN	10	160
Flatten	0	0
Dense	28x28x16 = 12,544	12,545
Output	0	0
Total		**12,705**

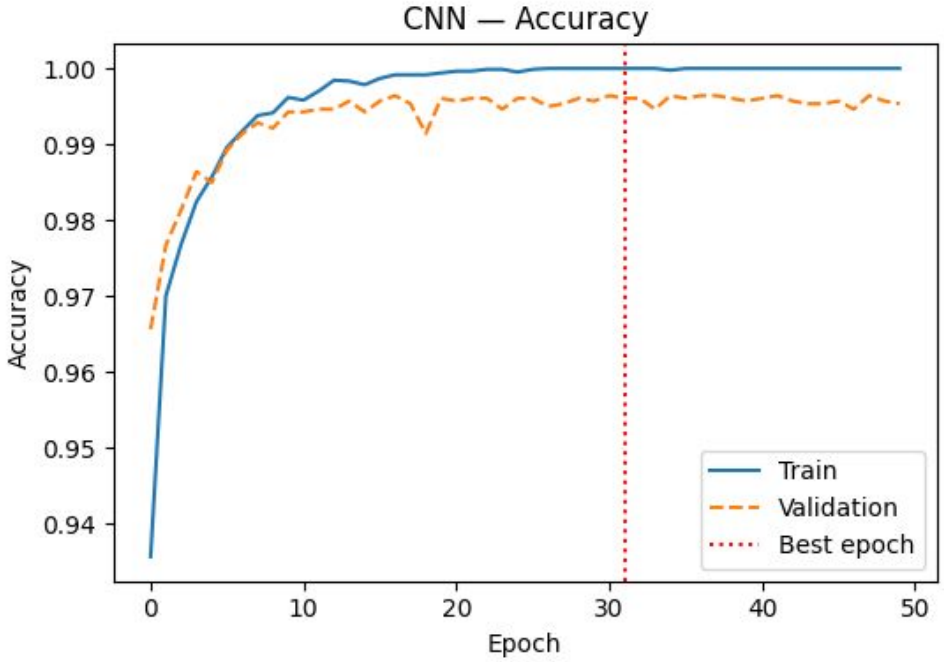


Convolutional Layer:
16 Filters
ReLU

Output Layer:
1 Node
Sigmoid

MLP and CNN Results





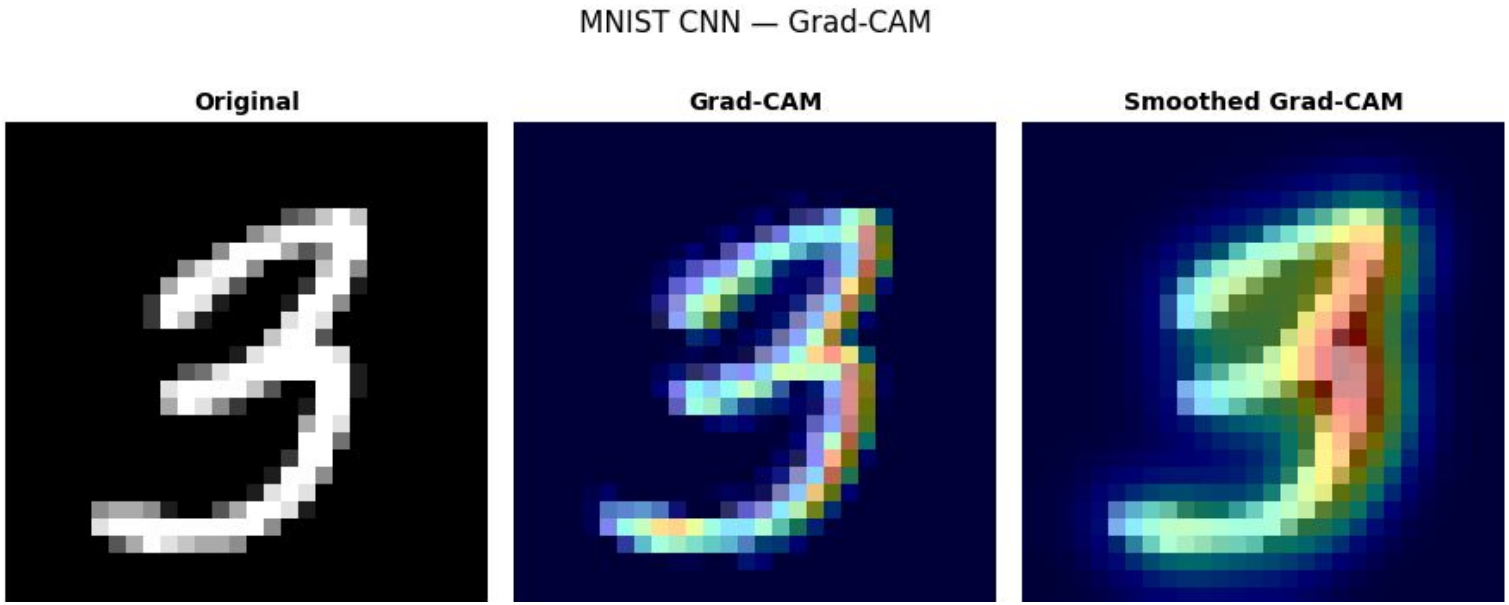
MLP test accuracy (held-out 20%): 0.9903
CNN test accuracy (held-out 20%): 0.9936

Class Activation Maps (CAMs)

Class Activation Mapping (CAM) is a technique used to visualize which specific regions of an image CNN focuses on to make a prediction. (Limited to certain CNN architectures).

Gradient-weighted Class Activation Mapping (Grad-CAM) generalizes this approach to any CNN. They calculate the gradients of the target class score with respect to the feature maps in a specific convolutional layer. These gradients are global-average-pooled to determine the "importance weight" of each feature map.

Grad-CAM: What makes a “3” a “3”?



These methods are far from perfect - but provide something for a human to look at and make a decision.

Opening up the lid on the black-box.

Biological MNIST

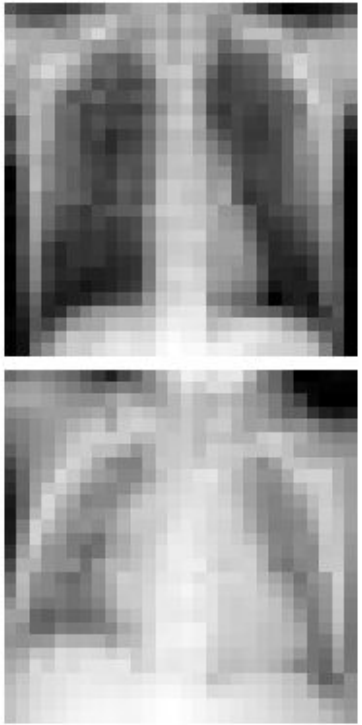
We apply the identical CNN architecture and Grad-CAM pipeline to a real biomedical dataset.

PneumoniaMNIST (MedMNIST v2; Yang et al. 2023) contains pediatric chest X-rays downsampled to 28 x 28, labelled Normal (y=0) or Pneumonia (y=1).

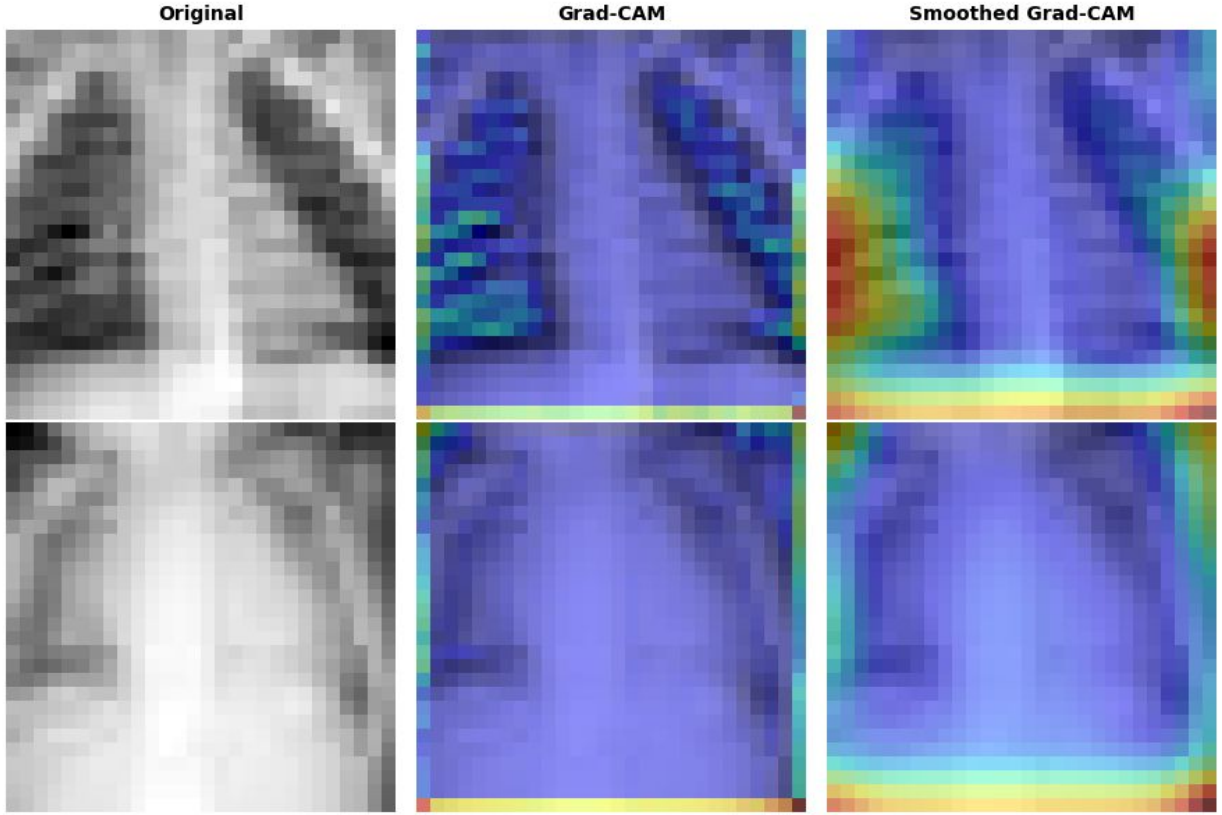
The dataset provides its own train/validate and test splits:

- Train accuracy: 97.7%
- Validation accuracy: 95.4%
- Test accuracy (held-out): 85.1%

The central question is no longer only accuracy, but: *does the network help us determine clinically relevant parts of the image?*



Biological MNIST with CAM



Deep Learning

SIAM LS Minitutorial #1

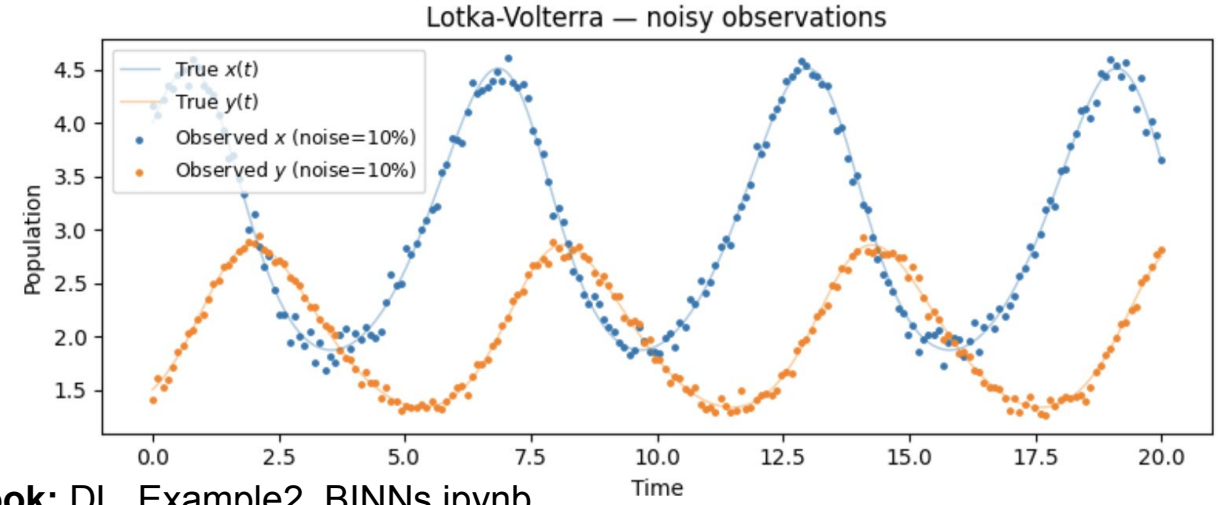
https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • Preventing Overfitting • Ex #2: Application
50 Mins	Deep Learning
	• Definitions • Ex #1: Classification with MNIST • **Ex #2: BINNs for Learning ODE Parameters**
10 Mins	Wrap-Up

Example 2: Fitting Data with BINNs

1) Two phase training of Biological Inspired Neural Networks (BINNs)
2) Getting interpretable information out of a BINN

Goal: How do I fit parameters in a Lotka-Volterra Model to noisy data



Python notebook: DL_Example2_BINNs.ipynb

What is a BINN

A **Biologically-Informed Neural Network (BINN)** combines a standard neural network with a mechanistic ODE model. Instead of just fitting the data, the network is penalized for violating the *known biology* - so training recovers both a smooth trajectory and interpretable biological parameters.

Q: Why use a BINN?

Ans: You know the equations but not the parameters, and your data is too noisy to trust classical fitting. A BINN uses the neural network to handle the noise, while the ODE constraint forces the learned curve to be consistent with your mechanistic model.

Two Phase: Training a BINN

Phase 1: Use a NN to generate a smooth representation of the data.

Learn the smooth version of the data at points you do not observe.

Phase 2: Add the ODE constraint and recovery the biological parameters.

Find the parameter that make the mechanistic equations hold along the smooth curve learned in Phase 1.

Two Phase: Training a BINN

Phase 1: Use a NN to generate a smooth representation of the data.

Learn the smooth version of the data at points you do not observe.

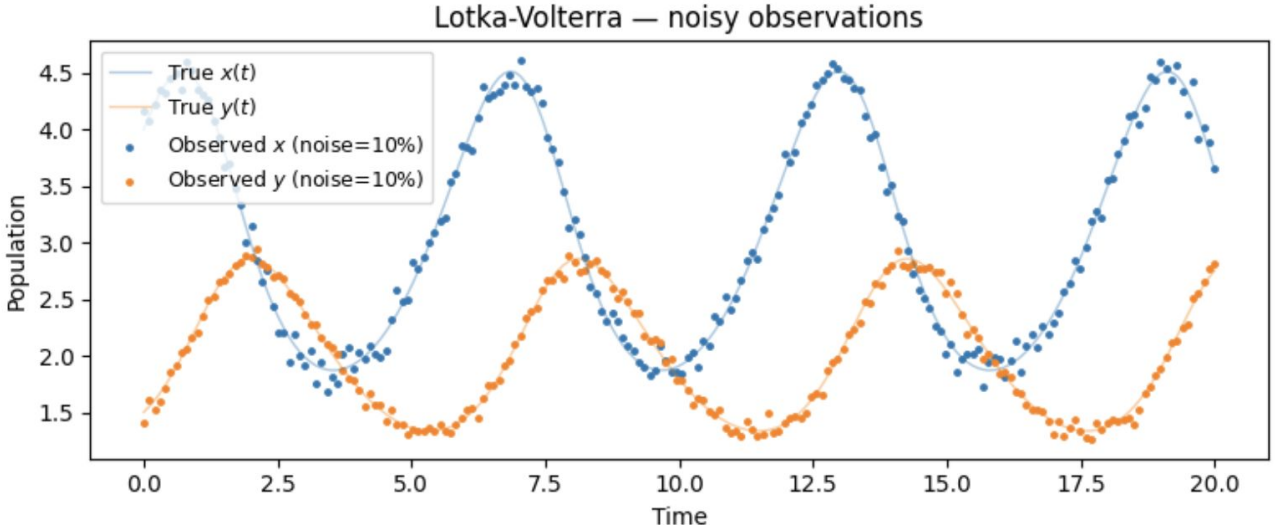
$$\text{Fit the Shape} \rightarrow \text{Loss}_1 = \text{Loss(Data)} + \lambda_{IC}\,\text{Loss(IC)}$$

Phase 2: Add the ODE constraint and recovery the biological parameters.

Find the parameter that make the mechanistic equations hold along the smooth curve learned in Phase 1.

$$\text{Fit the Shape \& Satisfy ODE} \rightarrow \text{Loss}_2 = \text{Loss(Data)} + \lambda_{BIO}\,\text{Loss(BIO)} + \lambda_{IC}\,\text{Loss(IC)}$$

Our Data & Model



$$\dot{x} = \alpha x - \beta xy$$

$$\dot{y} = \gamma xy - \delta y$$

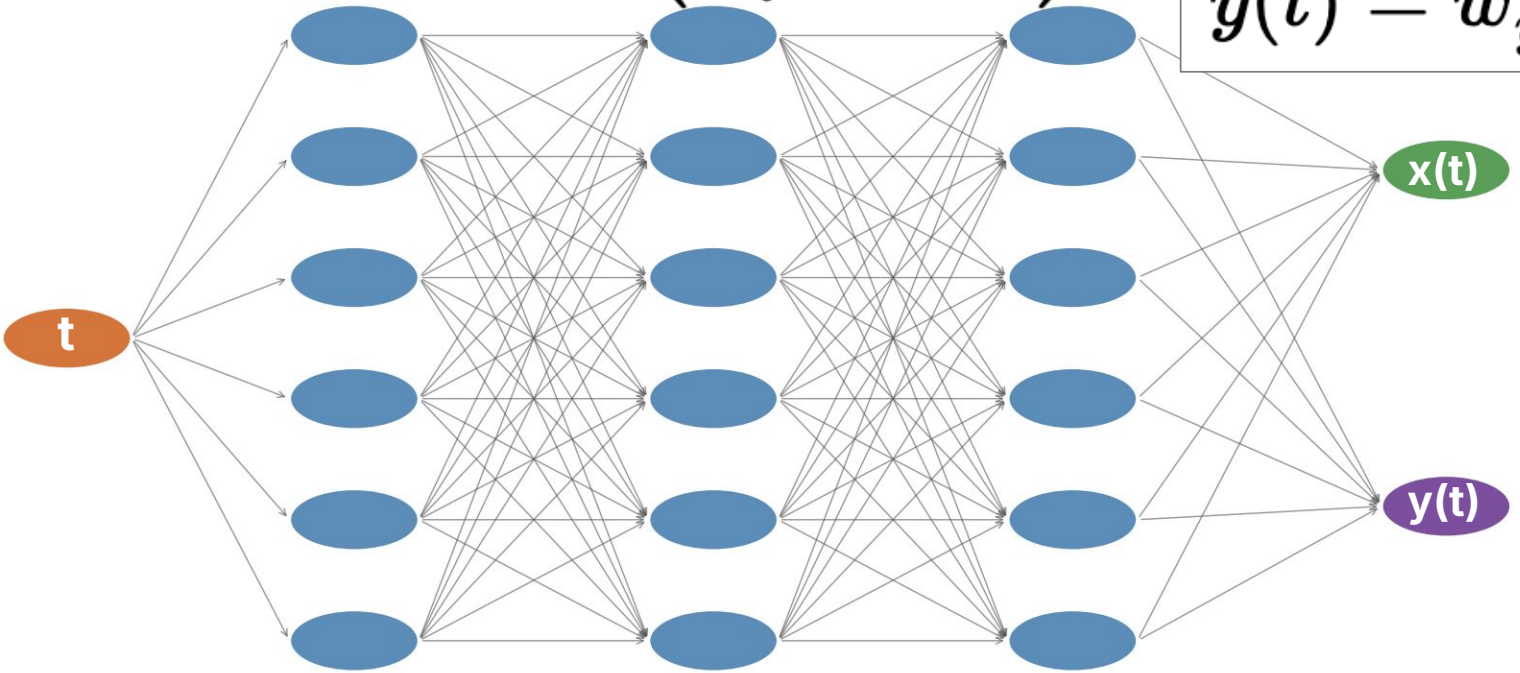
Parameter	True
alpha	1.2000
beta	0.6000
gamma	0.3000
delta	0.9000

BINN Architecture (Phase 1)

$$\hat{x}(t) = \vec{w}_x^T \vec{h} + b_x$$

$$\hat{y}(t) = \vec{w}_y^T \vec{h} + b_y$$

$$h_i = \tanh\left(\vec{w}_i^T \vec{x} + b_i\right)$$



Input Layer:
1 Node

3 Hidden Layers: 32 Nodes/Layer
Tanh Activation Function

Output Layer:
2 Nodes
Linear

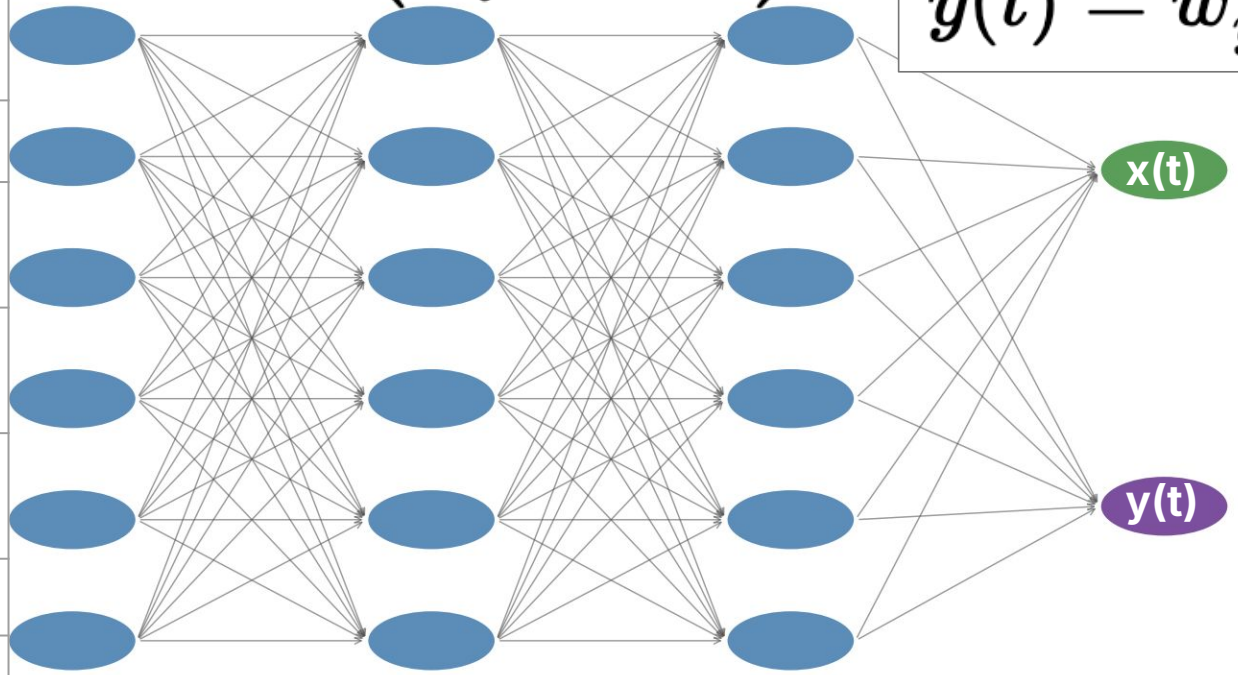
BINN Architecture (Phase 1)

$$\hat{x}(t) = \vec{w}_x^T \vec{h} + b_x$$

$$\hat{y}(t) = \vec{w}_y^T \vec{h} + b_y$$

$$h_i = \tanh\left(\vec{w}_i^T \vec{x} + b_i\right)$$

# of Parameters		
	Per Node	Per Layer
Input	0	0
Hidden 1	2	64
Hidden 2	33	1056
Hidden 3	33	1056
Output	33	66
Total		**2242**



Input Layer: 1 Node

3 Hidden Layers: 32 Nodes/Layer
Tanh Activation Function

Output Layer: 2 Nodes Linear

BINN Architecture (Phase 1 + Phase 2)

# of Parameters		
	Per Node	Per Layer
Input	0	0
Hidden 1	2	64
Hidden 2	33	1056
Hidden 3	33	1056
Output	33	66
Bio Param	**0**	**4**
Total		**2246**

$$\mathcal{L}_{\text{data}} = \frac{1}{N}\sum_{i=1}^{N}\left[(\hat{x}(t_i) - x_i^{\text{obs}})^2 + (\hat{y}(t_i) - y_i^{\text{obs}})^2\right]$$

$$\mathcal{L}_{\text{bio}} = \frac{1}{M}\sum_{j=1}^{M}\left[\left(\left.\frac{d\hat{x}}{dt}\right|_{\tau_j} - \alpha\hat{x}_j + \beta\hat{x}_j\hat{y}_j\right)^2 + \left(\left.\frac{d\hat{y}}{dt}\right|_{\tau_j} - \gamma\hat{x}_j\hat{y}_j + \delta\hat{y}_j\right)^2\right]$$

$$\mathcal{L}_{\text{ic}} = (\hat{x}(0) - x_0^{(1)})^2 + (\hat{y}(0) - x_0^{(2)})^2$$

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_{\text{bio}}\,\mathcal{L}_{\text{bio}} + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}}$$

- Classical methods: differentiate the **noisy data** → noisy derivatives → bad parameter estimates
- BINN: differentiate the **smooth network** → exact derivatives → clean parameter estimates.

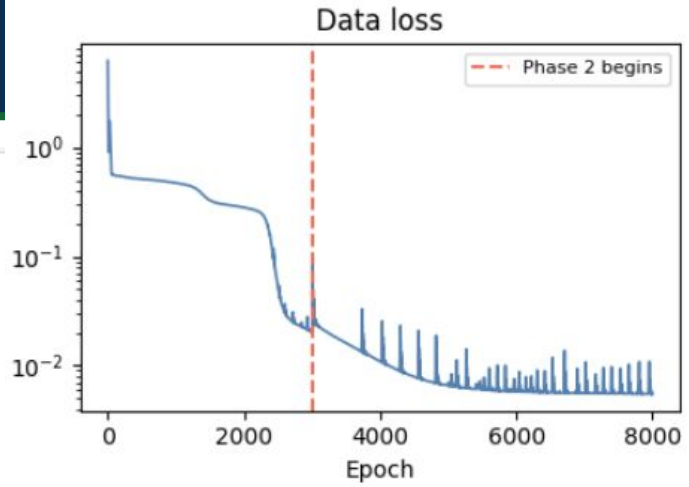
Results: Phase 1 and 2 of Training

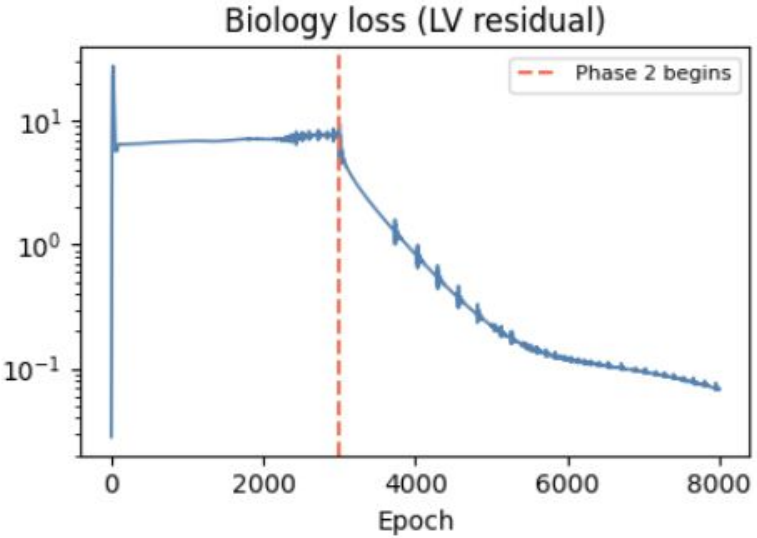
Phase 1/2 — data fit only (3000 epochs)

Epoch	data_loss	ic_loss
500/3000	0.5129	0.0000
1000/3000	0.4720	0.0000
1500/3000	0.3271	0.0000
2000/3000	0.2811	0.0000
2500/3000	0.0513	0.0000
3000/3000	0.0201	0.0000

Phase 2/2 — data + biology (5000 epochs, λ_bio=0.01)

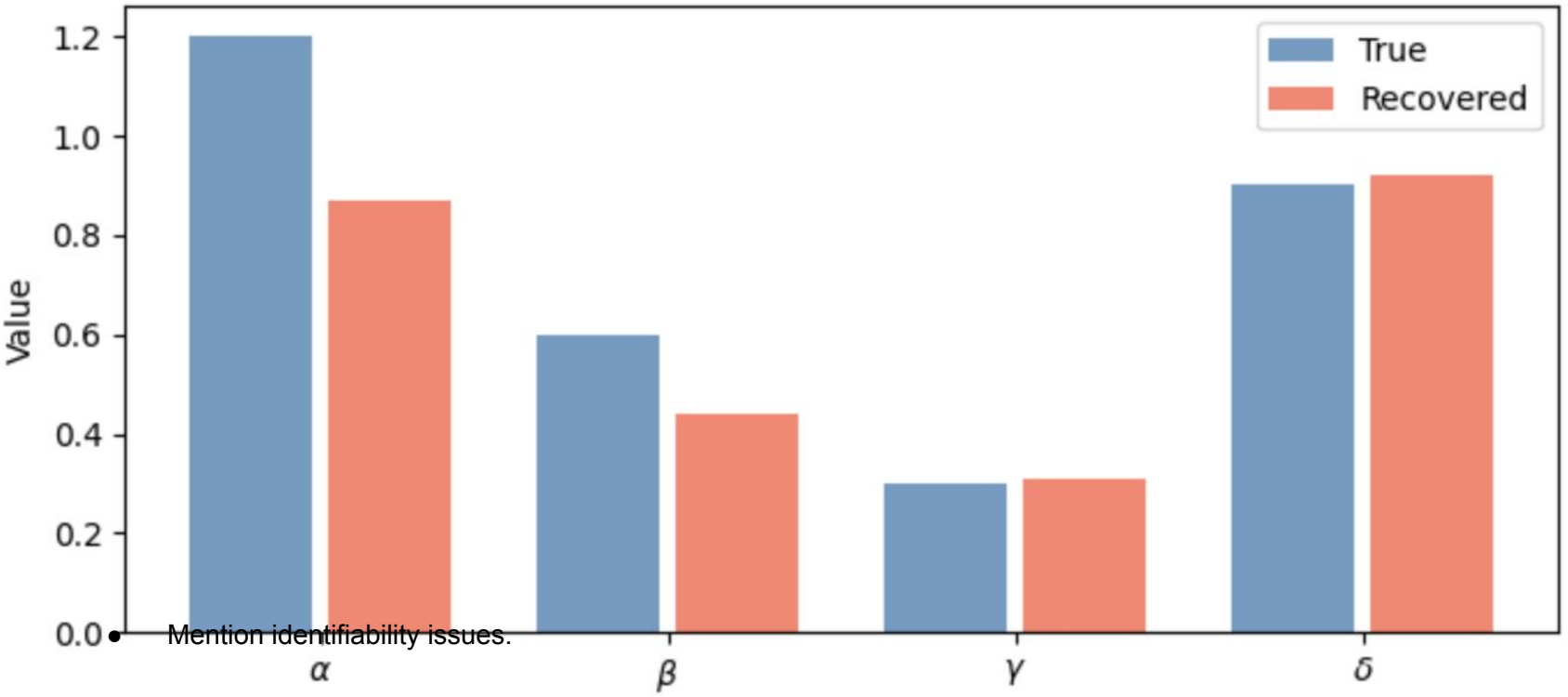
Epoch	data	bio	ic	α	β	γ
500/5000	0.0161	1.8686	0.0000	0.6263	0.4081	0.4007
1000/5000	0.0110	0.8397	0.0000	0.6830	0.3830	0.3650
1500/5000	0.0079	0.4062	0.0000	0.7155	0.3738	0.3425
2000/5000	0.0068	0.2281	0.0000	0.7332	0.3738	0.3276
2500/5000	0.0061	0.1496	0.0000	0.7458	0.3786	0.3178
3000/5000	0.0059	0.1180	0.0000	0.7601	0.3859	0.3121
3500/5000	0.0057	0.1054	0.0000	0.7787	0.3953	0.3093
4000/5000	0.0056	0.0942	0.0000	0.8027	0.4071	0.3082
4500/5000	0.0057	0.0816	0.0005	0.8326	0.4218	0.3079
5000/5000	0.0055	0.0685	0.0000	0.8688	0.4394	0.3078





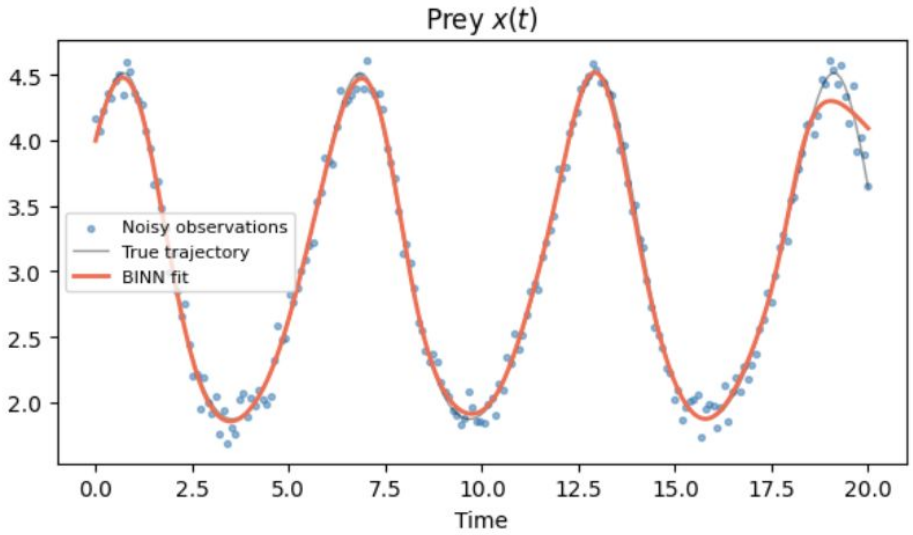
Results: Parameter Accuracy

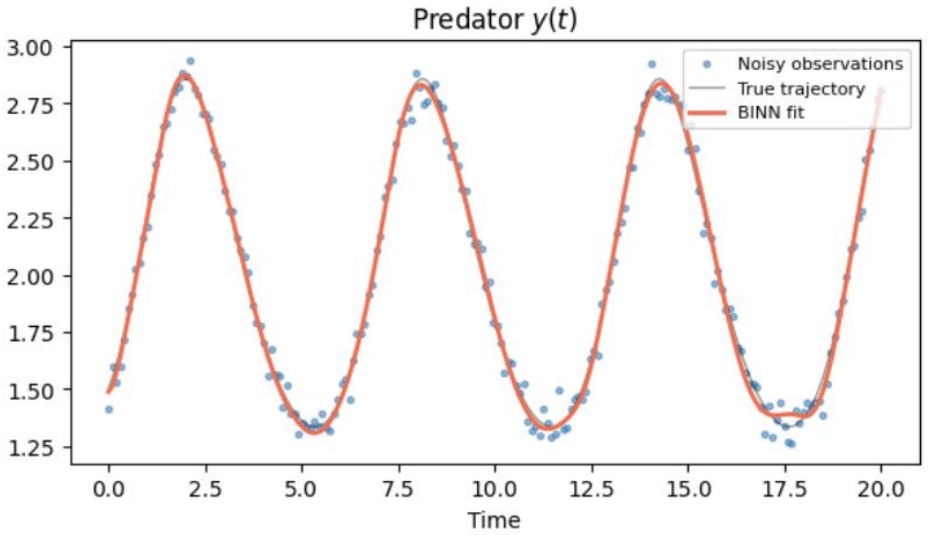
Parameter	Recovered	True	Error
alpha	0.8688	1.2000	27.6%
beta	0.4394	0.6000	26.8%
gamma	0.3078	0.3000	2.6%
delta	0.9186	0.9000	2.1%



- Mention identifiability issues.

Results: Trajectory Matching

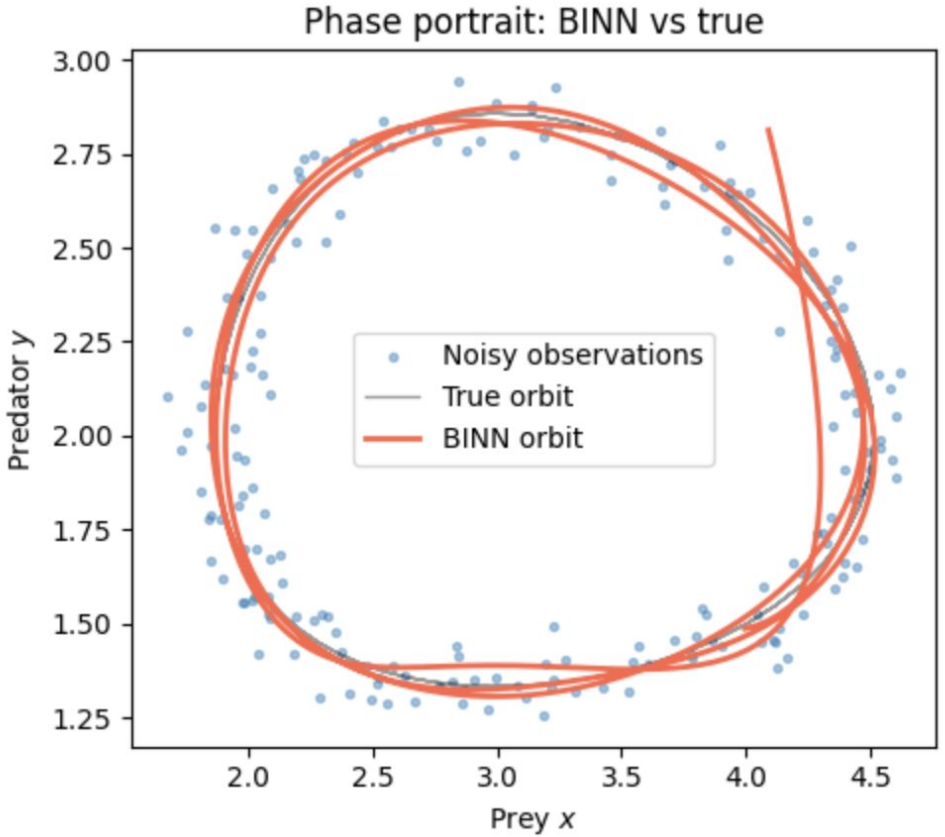




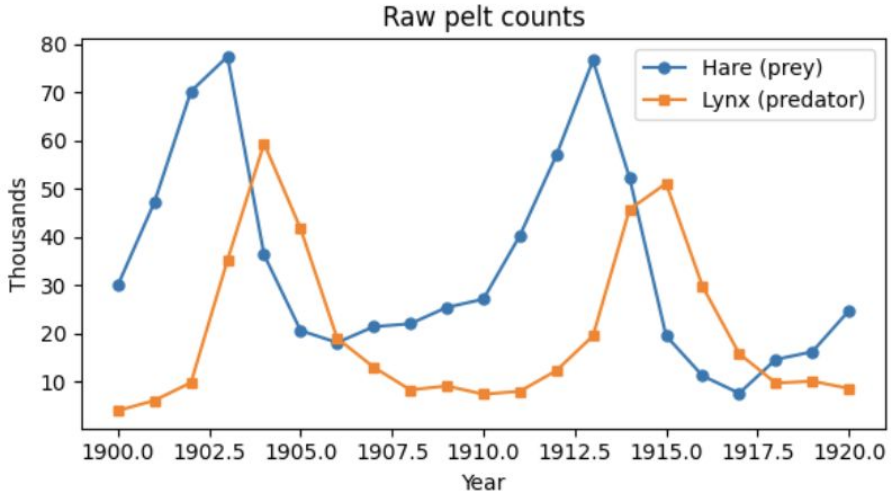
- We recover smooth trajectories….
- These fit our data well - but not perfectly
- We can exactly take derivatives

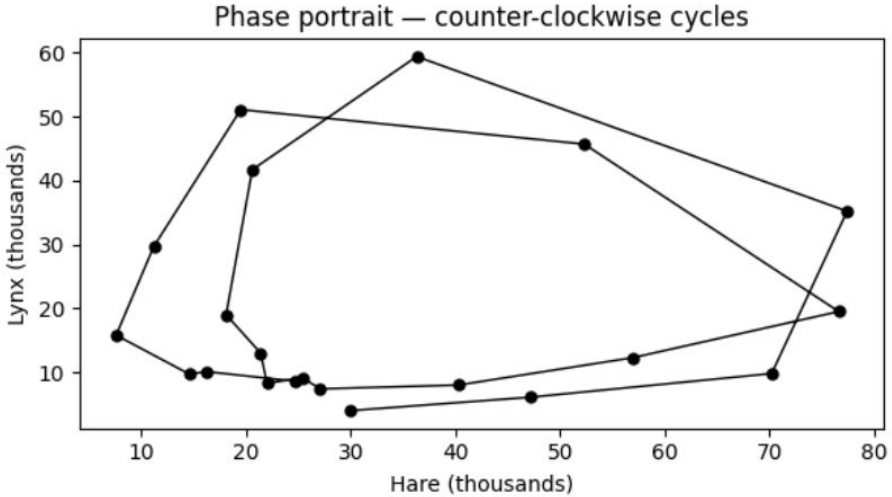
Results: Phase Portrait

- Qualitatively similar behavior
- Reconstruction closes (looks periodic)



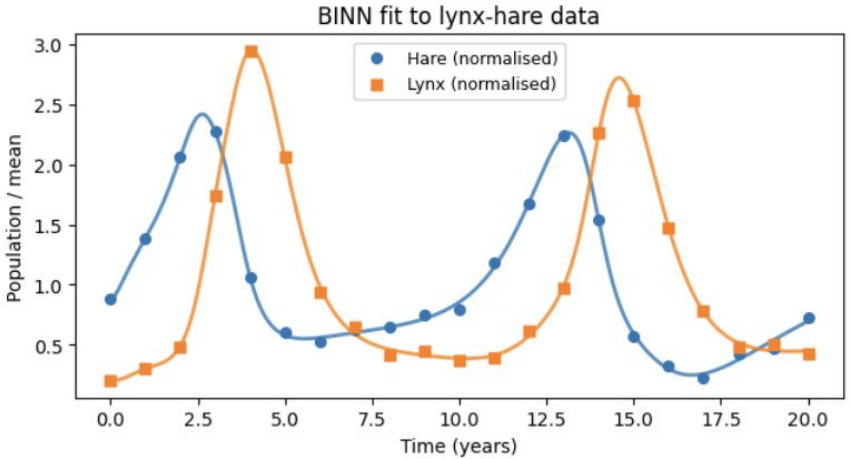
Real Data: Hudson's Bay Company Lynx–Hare Data

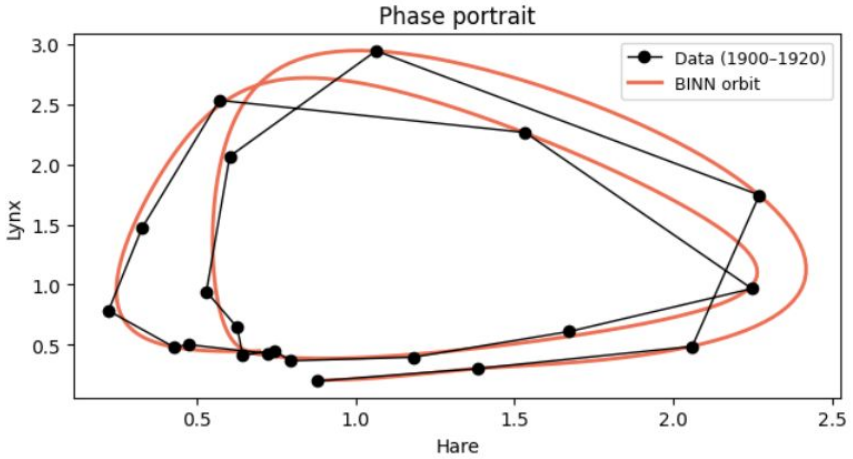




- Records of annual pelt counts (furs traded) collected by the Hudson's Bay Company in Canada, spanning roughly 1845–1935 (we use 1900–1920)
- Counts are a proxy for population size
- One of the most famous datasets in mathematical biology — the canonical real-world example of Lotka-Volterra dynamics
- Perfect test case for BINNs: real, noisy, biological time series w/ mechanistic model

Real Data: Hudson's Bay Company Lynx–Hare Data





- Similar behavior recovered to the real data!
- Data normalization and other details were important for performance.

BINNs Summary

Pros:

- Handles sparse and noisy data
- Derivatives are computed exactly through the network, not from noisy measurements
- No ODE solver calls during training
- Recovers interpretable biological parameters directly from training
- Naturally extends to irregular sampling and unknown initial conditions

Cons:

- You must know the equation structure
- Hyperparameters need needs tuning
- Sensitive to initial conditions and population scale
- Slower and harder to train than classical fitting when data is clean and plentiful
- No uncertainty quantification

Use BINNs when…

- Data is sparse or irregularly sampled.
  - Bayesian methods/MCMC still needs to call solve IVP at every sample)
- The ODE is stiff or expensive to solve.
  - Bayesian MCMC becomes prohibitively slow
- You want gradient-based optimization end-to-end
- You have PDEs or spatiotemporal data

BINNs win on scalability and flexibility; MCMC wins on uncertainty quantification.

Wrap-Up

SIAM LS Minitutorial #1

https://github.com/ssindiUCM/SIAM-LS26-MT1

Time	Event
10 Mins	Overview & Orientation
50 MIns	Traditional Machine Learning
	• Definitions • Ex #1: Binary Classification • Preventing Overfitting
50 Mins	Deep-Learning
	• Definitions • Ex #1: Classification with MNIST • Ex #2: BINNs for Learning ODE Parameters
10 Mins	Wrap-Up

Learning From Data

This tutorial covers a single question: what can I learn from the data I have?

The answer depends on: what do I know?

- Do I know the model/mechanism/equation structure?
- What does my data look like? (cross-sectional, time series, images)
- What am I trying to predict? (class label, continuous value, ODE parameters)

Choosing Your Method

	Know (or assume) the equations	Do not know the equations *pure data-driven*
Static / cross-sectional data	Linear models, logistic regression MLE fits known model form to static observations	Random forests, deep learning Let the model discover structure from data
Time series / dynamical data	Mechanistic models (ODEs/PDEs) Fit via MLE, MCMC, or BINNs	SINDy, DSO, Neural ODEs Discover DE from time series

Naturally Interpretable!

Choosing Your Method (*Interpretability Emerging)*

	Know (or assume) the equations	Do not know the equations *pure data-driven*
Static / cross-sectional data	Linear models, logistic regression MLE fits known model form to static observations	Random forests, deep learning Let the model discover structure from data GradCAMs
Time series / dynamical data	Mechanistic models (ODEs/PDEs) Fit via MLE, MCMC, or BINNs	SINDy, DSO, Neural ODEs Discover DE from time series Explicit equations learned

Naturally Interpretable!

Closing Thoughts

- Interpretability is a spectrum!
- There are lots of details that we left out (i.e,. Hyperparameter tuning)
- Each of these methods has a host of options that you may consider and modify based on your specific context.
  - The best choices depend on the dataset, the research question, and the goal of the analysis.
- Model building is an ongoing process, and there is still much to learn.
- The methods shown here provide a starting point, but they can be adapted to fit different contexts.

Thank you!

References

- Bishop, C. M. Pattern Recognition and Machine Learning. Springer, 2006.
- Brunton & Kutz (2022). Data-Driven Science and Engineering. Cambridge. (Covers SINDy, PINNs, and equation learning in a unified framework.)
- Hastie, T., Tibshirani, R., & Friedman, J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer, 2009.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. An Introduction to Statistical Learning: with Applications in R (2nd ed.). Springer, 2021.
- Kuhn, M., & Johnson, K. Applied Predictive Modeling. Springer, 2013.
- Kuhn, M., & Johnson, K. Feature Engineering and Selection: A Practical Approach for Predictive Models. CRC Press, 2019.
- Lagergren et al. (2020). Biologically-informed neural networks guide mechanistic modeling from sparse experimental data. PLOS Computational Biology.
- Raissi, Perdikaris & Karniadakis (2019). Physics-informed neural networks. Journal of Computational Physics. [paper]
- Rockafellar, R. T. Convex Analysis. Princeton University Press, 1970.
- Ruopp, M. D., Perkins, N. J., Whitcomb, B. W., & Schisterman, E. F. *Youden Index and Optimal Cut-Point Estimated from Observations Affected by a Lower Limit of Detection.* *Biometrical Journal*, **50**(3), 419–430, 2008.
- Sanderson, G. *But what is a Neural Network? (Deep Learning Chapter 1)*. 3Blue1Brown, 2017. https://www.3blue1brown.com/lessons/neural-networks
- Shrestha, P. Exploring the Breast Cancer Dataset: Do More Features Always Mean Better Results? Medium, 2025.
- Sidey-Gibbons, M., & Sidey-Gibbons, J. A. Machine Learning in Medicine: A Practical Introduction. BMC Medical Research Methodology, 19, 64, 2019.
- Steyerberg, E. W. Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating (2nd ed.). Springer, 2019.
- van Smeden, M., de Groot, J. A. H., Moons, K. G. M., Collins, G. S., Altman, D. G., Eijkemans, M. J. C., & Reitsma, J. B. No Rationale for 1 Variable per 10 Events Criterion for Binary Logistic Regression Analysis. BMC Medical Research Methodology, 16, Article 163, 2016.
- Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Pfister, H., & Ni, B. MedMNIST v2: A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification. *Scientific Data*, **10**, 41, 2023.